

Curated File Guide: server.mjs

The local backend that saves drafts, runs compilers, checks GitHub publishing access, imports/pushes site files, handles updates, and relays AI.

Chapter 1: Server Abstract, Background, And Design Purpose

This chapter explains server.mjs as a complete backend file before isolating any single function. The useful first question is not 'what does line one do?' but 'what job does this file perform inside the whole application?' The answer is that server.mjs is the private local backend for the OMB Portfolio Builder. It receives browser requests from the builder UI, performs privileged work on the owner's machine, and returns structured results that the UI can display.

The file exists because a portfolio builder needs powers that a normal webpage should not have. A browser page can collect a project title, display an editor, and send a request. It should not directly write arbitrary files, run compilers, launch installers, push to GitHub, or store publishing authorization. server.mjs becomes the controlled layer between the friendly interface and those dangerous machine-level operations.

Analogy: *Think of the builder UI as the front desk of a hardware lab. The visitor can ask for a test, a file save, a compile, or a website push. server.mjs is the lab technician behind the counter. It knows which cabinet the files belong in, which tools are allowed, which requests are local, and which operations need authentication before anything public changes.*

Abstract

server.mjs is the local nervous system of the OMB Portfolio Builder. It is not the public website and it is not the Electron shell. It is the private backend that runs on the owner's machine and gives the builder the powers a browser page should not have by itself: reading and writing local files, saving drafts, accepting uploads, running compilers, using Git, checking publishing authorization, importing a target site, launching installers, and relaying AI requests. The file is intentionally broad because the builder is local-first. Instead of depending on a hosted database and a remote application server, the desktop app starts this Node server beside the user-owned workspace and talks to it through local API endpoints.

The most important idea is boundary control. The frontend can draw buttons and collect input, but it cannot safely decide where files are written, which Git repository receives a push, whether a command should run, or whether a request came from the local computer. server.mjs owns those decisions. It is the layer that turns a UI action into a guarded filesystem, compiler, Git, installer, or AI operation.

Introduction

When the builder starts, Electron loads a local page. That page needs catalog data, templates, upload handling, compile actions, publishing actions, and update checks. A static HTML page cannot perform those operations directly without either exposing dangerous privileges or running into browser security walls. server.mjs solves that by becoming a small local application server. It serves the builder files, exposes API endpoints under /api, and keeps privileged work in Node where the filesystem, process runner, and Git commands are available.

A useful mental model is to treat this file like a workshop manager. The frontend is the workbench surface where the user sees projects, editors, buttons, and dialogs. server.mjs is the person behind the counter who knows where the tools are, where files belong, whether a customer has permission to ship something, and whether the requested operation is safe. If the frontend says 'save this PDF under this project,' server.mjs sanitizes the path and writes it under the correct docs folder. If the frontend says 'compile this C++ program,' server.mjs finds g++, creates a temporary run folder, captures terminal output, and returns a result object. If the

frontend says 'apply to site,' server.mjs checks Git authorization first, then synchronizes publishable files and pushes them.

Background Information

The file uses built-in Node modules rather than a large web framework. `createServer` from `node:http` accepts requests, `node:fs/promises` handles files, `node:child_process` runs compilers and Git, `node:path` normalizes paths, `node:crypto` hashes authorization cache scopes, and `node:os` identifies the current machine. This keeps the backend predictable and package-light. The tradeoff is that routing and request handling are written manually. That is why `handleApi` is large: it plays the role that Express or another framework would normally play.

There are three important roots. `root` is the builder workspace that contains the site files and local draft catalog. `portfolioRoot` is the publish mirror, which may be separate from `root` so generated website files can be pushed without mixing in builder-only files. `compileRoot` is where project-local code workspaces and temporary compiler run folders live. Most path-related functions exist to make sure a request stays inside one of these roots.

The backend is also intentionally local-sensitive. Some endpoints are read-only, but write endpoints reject non-local callers. That is why functions such as `isLocalRequest` matter: the builder may be opened in a browser or accessed on a LAN during testing, but operations that change files, compile code, install tools, or publish should be performed only from the owner's computer.

Purpose Of The File

The purpose of `server.mjs` is to make the builder operational instead of decorative. Without it, `template-preview.html` and `template-preview.js` would still be able to render a screen, but they would not be able to reliably save drafts, upload project evidence, run compilers, authenticate GitHub targets, import an existing website, launch updates, or call private AI providers. The file therefore holds the application's real side effects.

The file is also a protection layer. It validates filenames, normalizes repository URLs, keeps publish authentication scoped to a machine and target, prevents stageable publishing paths from wandering outside the publish mirror, blocks remote write requests, caps text fetched from external sources, and strips terminal output into UI-safe strings. Those decisions are not cosmetic. They prevent the portfolio builder from becoming a general-purpose remote command runner or a tool that accidentally pushes the wrong files.

Primary Responsibilities

The first responsibility is serving local files. The same server that handles `/api` requests also serves `index.html`, `styles.css`, `script.js`, `template-preview.html`, project catalogs, assets, and uploaded docs during local preview. The static-serving logic checks that the requested path stays inside the workspace before reading the file. That simple check is what prevents a URL from escaping into arbitrary folders on the machine.

The second responsibility is compilation. Compile Code works because the backend can write source files to a project workspace, discover tools such as `node`, `python`, `gcc`, `g++`, `javac`, `iverilog`, `vvp`, and `LTspice`, run those tools with timeouts, collect `stdout` and `stderr`, cache successful builds, and return terminal output immediately to the frontend. The backend treats each language differently because JavaScript, Python, C, C++, Java, Verilog, SystemVerilog, LTspice, and HTML do not build the same way.

The third responsibility is publishing. Publishing is not just 'write projects.json.' The backend must verify that the publish mirror is a Git repository, know its origin remote and branch, check compatibility with a static portfolio site, synchronize with the remote to avoid non-fast-forward errors, stage only approved publish paths, commit if there are changes, and push to the selected branch. This is why the publishing code is careful and verbose.

The fourth responsibility is intelligence. For local AI, the backend can use an OpenAI API key or a local Ollama model. It also enriches portfolio context by reading public project links, same-site files, GitHub

repositories, GitHub profiles, README files, and selected source files when the question calls for it. The browser cannot safely hold private API keys, so `server.mjs` keeps that path private.

The fifth responsibility is application maintenance. The update functions check GitHub Releases, download an installer, write a PowerShell handoff script, close the running app, run the installer, and relaunch the updated executable. This is backend work because the browser cannot replace the running desktop app.

Important Data Objects And Why They Matter

`compileLanguageProfiles` is the backend's language map. It tells the compiler system what default filename to use, which file extensions belong to each language, what human label to show, which tools are required, and which Winget package can install the tool if automatic installation exists. Without this object, `compileAndRunCode` would need hardcoded language assumptions scattered throughout the file.

`compileToolCandidates` is the tool-discovery map. It lists command names and likely Windows install paths for Git-adjacent and compiler-adjacent executables. `findTool` uses this object so the app can work even when a tool is installed but not perfectly exposed on PATH. The caches `compileToolCache` and `compileToolVersionCache` exist so repeated compiles do not waste time rediscovering the same executables.

`publishPaths` is the publishing allowlist. It lists the site files and folders the backend is allowed to copy into the publish mirror and stage in Git. This is one of the most important safety objects in the file. Without an allowlist, an Apply to site operation could accidentally stage builder-only files, private drafts, temporary compile output, or unrelated local files.

`portfolioAiInstructions` is the system behavior contract for the portfolio assistant. It tells the model how to distinguish general conversation, general engineering, general knowledge, and portfolio-specific questions. It also tells the model when to use context, when not to invent, and how to handle public GitHub source excerpts. That string is long because it is the policy boundary for the AI feature.

Chapter 2: JavaScript And Node Syntax Foundations

This chapter is intentionally general. It explains the JavaScript and Node ideas needed to read the later server chapters, without tying the explanation to any one `server.mjs` function. Once these ideas are in place, the later chapters can focus on behavior instead of repeating syntax notes.

2.1 Modules And Toolboxes

Modern Node files can use ES module import statements. An import is a declaration that says, 'this file needs a tool that lives somewhere else.' The imported tool might come from Node itself, such as an HTTP server, path helper, cryptographic hash function, filesystem API, or child-process runner. In a local backend, these imports are the toolboxes that make the file more powerful than browser JavaScript.

Generic ES module import pattern

```
import { usefulTool } from "node:some-module";

const result = usefulTool("input");
```

The analogy is a lab bench. The bench itself is the file. The import statements are labeled instruments placed on the bench before work begins. Later functions do not need to manufacture a multimeter, oscilloscope, or power supply every time they need one; they reach for the prepared instrument.

2.2 Objects, Arrays, And Destructuring

JavaScript objects store named fields. They are common in this backend because API requests, compiler profiles, Git status reports, update reports, and AI context packets all carry several related values at once. Destructuring is simply a shortcut for pulling named fields out of an object so the function can work with them directly.

Generic object, destructuring, and array pattern

```
const profile = { language: "C", compiler: "gcc", timeoutMs: 30000 };
const { language, compiler } = profile;

const files = ["main.c", "filter.h", "filter.c"];
```

A good circuit analogy is a connector with labeled pins. The object is the connector body. Each property name is a pin label. Destructuring is grabbing only the pins you need for the next operation.

2.3 Async Work And Await

Node backends constantly wait for slow work: reading a file, writing a file, calling GitHub, running Git, launching a compiler, or waiting for a model response. JavaScript represents those delayed results with Promises. The `async` keyword lets a function use `await` inside its body. `await` pauses that function until the operation finishes, while the rest of Node can continue handling other work.

Generic asynchronous file operation

```
async function saveText(filePath, text) {
  await writeFile(filePath, text, "utf8");
  return { ok: true, filePath };
}
```

The lab analogy is waiting for a soldering iron to heat. The instruction is still part of the procedure, but the technician does not pretend the result exists before the tool is ready.

2.4 HTTP Requests And Responses

A local backend receives HTTP requests. Each request has a method, such as GET or POST, and a path, such as `/api/catalog`. A response has a status code, headers, and a body. Headers describe how the browser should interpret the body. The body carries the useful information, often JSON.

Generic request and JSON response pattern

```
if (request.method === "GET" && url.pathname === "/api/example") {
```

```
response.writeHead(200, { "Content-Type": "application/json" });
response.end(JSON.stringify({ ok: true }));
}
```

Read routing code like a switchboard. The method and path select the circuit. The selected branch decides which subsystem receives the request and what response returns to the browser.

2.5 Paths, Processes, And Guardrails

A backend that writes files must treat paths carefully. User input should never be allowed to decide an unrestricted disk location. The normal pattern is to start from a trusted root folder, combine safe path segments, normalize the result, and reject anything that escapes the root.

Generic path-boundary pattern

```
const target = path.resolve(rootFolder, safeSegment);
if (!target.startsWith(rootFolder)) {
  throw new Error("Path escaped the workspace.");
}
```

Process execution has a similar rule. A compiler or Git command should run with a specific executable, exact arguments, a working folder, a timeout, and captured output. That keeps the backend from becoming an open-ended command prompt.

Chapter 3: Runtime Boundaries, Response Shape, And File Roots

The first backend concern is boundary control. Before the app can compile code or push a website, it must know which folders are trusted, how responses are shaped, and how requests become safe data.

Important working state

These objects and constants are shown before the functions because they are the shared state the functions read from or protect. They are not syntax trivia; they explain what the backend remembers while it runs.

root

root is the trusted builder workspace. Almost every local file path eventually relates back to this folder, so it acts like the mechanical chassis of the backend. A request may name a file, but the backend decides whether that name can safely resolve under root.

Server state excerpt: root

```
const root = path.dirname(fileURLToPath(import.meta.url));
```

portfolioRoot

portfolioRoot is the publishing mirror. It can be the same folder as root, but the newer architecture lets it be separate so public website files can be synchronized without mixing in private builder-only files.

Server state excerpt: portfolioRoot

```
const portfolioRoot = path.resolve(process.env.OMB_PORTFOLIO_WORKSPACE || root);
```

compileRoot

compileRoot is the code-workspace area. Compile Code writes sources, cache folders, run folders, generated binaries, Java classes, HDL simulations, and VCD files here instead of scattering temporary build material through the portfolio.

Server state excerpt: compileRoot

```
const compileRoot = path.resolve(process.env.OMB_CODE_WORKSPACE || path.join(path.dirname(root), "compile-code"));
```

types

types is the small static-file MIME table. It tells the browser whether a served file is JavaScript, CSS, JSON, an image, a PDF, or a generic download. Without it, local preview could serve files as the wrong content type.

Server state excerpt: types

```
const types = {
  ".css": "text/css",
  ".csv": "text/csv",
  ".html": "text/html",
  ".ico": "image/x-icon",
  ".js": "application/javascript",
  ".json": "application/json",
  ".md": "text/markdown",
  ".pdf": "application/pdf",
  ".png": "image/png",
  ".jpg": "image/jpeg",
  ".jpeg": "image/jpeg",
  ".svg": "image/svg+xml",
  ".txt": "text/plain",
  ".webp": "image/webp",
  ".xml": "application/xml",
  ".xlsx": "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
  ".zip": "application/zip"
};
```

draftPath

draftPath points at the private local draft catalog. Save draft writes here so the builder can preserve unfinished work without immediately changing the public projects.json.

Server state excerpt: draftPath

```
const draftPath = path.join(root, "projects.local.json");
```

catalogPath

catalogPath points at the publish-ready catalog. Apply to site writes here before the publishing path copies and commits website files.

Server state excerpt: catalogPath

```
const catalogPath = path.join(root, "projects.json");
```

publishPaths

publishPaths is the publishing allowlist. It is one of the most important safety objects in the file because it names exactly what can leave the builder workspace and enter the public website repository.

Server state excerpt: publishPaths

```
const publishPaths = [  
  "projects.json",  
  "docs",  
  "assets",  
  "index.html",  
  "styles.css",  
  "script.js",  
  "electronics-search.js",  
  "Backgrounds",  
  ".nojekyll",  
  "_headers",  
  ".well-known",  
  "robots.txt",  
  "sitemap.xml",  
  "CNAME"  
];
```

Function walkthrough

securityHeaders

securityHeaders is a small function, but it sits at the outer skin of the backend. The function returns a headers object that every local response can reuse. It begins with fixed browser-safety headers, then merges the caller's extra headers at the end.

Full source for securityHeaders

```
function securityHeaders(extra = {}) {  
  return {  
    "X-Content-Type-Options": "nosniff",  
    "Referrer-Policy": "strict-origin-when-cross-origin",  
    "Permissions-Policy": "camera=(), microphone=(), geolocation=(), payment=(), usb=()",  
    "Cross-Origin-Opener-Policy": "same-origin",  
    ...extra  
  };  
}
```

Reading the body is straightforward: the returned object disables MIME sniffing, limits referrer leakage, blocks browser permissions that the builder does not need, and keeps the opener policy same-origin. The spread of extra is intentionally last so a caller can add Content-Type, Cache-Control, or another endpoint-specific header without rebuilding the safety baseline.

This helper is not included because the syntax is difficult. It is included because repeated headers are easy to forget. Centralizing them makes local API responses and static-file responses behave consistently.

sendJson

sendJson is the backend's normal API response writer. The caller gives it the Node response object, a status code, and a data object. The function writes headers first, then serializes the data as formatted JSON and ends the response.

Full source for sendJson

```
function sendJson(response, status, data) {
```

```

response.writeHead(status, {
  ...securityHeaders(),
  "Content-Type": "application/json",
  "Cache-Control": "no-store, no-cache, must-revalidate"
});
response.end(JSON.stringify(data, null, 2));
}

```

The important sequence is header policy before body output. It calls `securityHeaders`, adds `application/json`, disables caching, and only then sends `JSON.stringify(data, null, 2)`. The indentation is not required by machines, but it makes API responses readable during debugging.

When later endpoint branches call `sendJson`, they do not need to remember the JSON content type or cache rule. That keeps `handleApi` focused on routing decisions instead of response formatting.

readRequestJson

`readRequestJson` turns a POST body stream into a parsed object. Node gives request data in chunks; the function collects those chunks, rejects bodies above a fixed size, and parses the final string as JSON.

Full source for readRequestJson

```

async function readRequestJson(request) {
  const chunks = [];
  let size = 0;

  for await (const chunk of request) {
    size += chunk.length;
    if (size > 60 * 1024 * 1024) {
      throw new Error("Request body is too large.");
    }
    chunks.push(chunk);
  }

  return JSON.parse((Buffer.concat(chunks).toString("utf8") || "{}").replace(/^\\uFEFF/, ""));
}

```

The chronological flow matters. It starts with an empty body, appends each incoming chunk, throws if the body grows past sixty megabytes, and waits for the stream to finish. Only after the stream is complete does it call `JSON.parse`. That order prevents partially received JSON from being treated as a real command.

This function is the front door for almost every write action. Save draft, upload, compile, authenticate target, apply to site, and AI all depend on request data arriving as a safe object instead of an uncontrolled stream.

resolveInsideRoot

`resolveInsideRoot` converts caller-supplied path segments into a path under the builder workspace. It resolves the final path, compares it against root, and rejects the request if the path escapes the workspace.

Full source for resolveInsideRoot

```

function resolveInsideRoot(...segments) {
  const target = path.normalize(path.join(root, ...segments));
  if (!target.startsWith(root)) {
    throw new Error("Resolved path is outside the workspace.");
  }
  return target;
}

```

The key detail is that `path.normalize` and `path.join` collapse the requested segments before comparison. That means dangerous segments such as `'..'` are interpreted before the safety check. The function then returns the normalized path only if it still belongs under root.

This is the filesystem equivalent of a current limit on a bench supply. The UI can request a destination, but this function keeps the request inside the safe operating region.

resolveInsidePortfolioRoot

`resolveInsidePortfolioRoot` performs the same boundary check for the publish mirror. It exists separately from `resolveInsideRoot` because the public portfolio workspace can live outside the builder workspace.

Full source for resolveInsidePortfolioRoot

```

function resolveInsidePortfolioRoot(...segments) {

```

```

const target = path.resolve(portfolioRoot, ...segments);
const relative = path.relative(portfolioRoot, target);
if (relative.startsWith("..") || path.isAbsolute(relative)) {
  throw new Error("Resolved path is outside the portfolio workspace.");
}
return target;
}

```

The function first builds the candidate path from `portfolioRoot` and the supplied segments. If the resolved path does not start inside `portfolioRoot`, it throws. Publishing then receives a usable path or no path at all; there is no middle state where a bad path is silently accepted.

resolveInsideCompileRoot

`resolveInsideCompileRoot` applies the boundary rule to code workspaces. Compile Code writes source files, run folders, binary artifacts, Java classes, and waveform files, so it needs the same path discipline as uploads and publishing.

Full source for resolveInsideCompileRoot

```

function resolveInsideCompileRoot(...segments) {
  const target = path.resolve(compileRoot, ...segments);
  const relative = path.relative(compileRoot, target);
  if (relative.startsWith("..") || path.isAbsolute(relative)) {
    throw new Error("Resolved path is outside the compile workspace.");
  }
  return target;
}

```

This guard is especially important because filenames and project ids can originate from user-edited project data. The function ensures a source filename becomes a file inside `compileRoot`, not an accidental write to another part of the computer.

Chapter 4: Compile Code Workspace And Tool Execution

This chapter follows the backend IDE path. The UI sends code and project workspace files; server.mjs turns that material into safe filenames, real source files, compiler invocations, terminal output, cached artifacts, and HDL scope data.

Important working state

These objects and constants are shown before the functions because they are the shared state the functions read from or protect. They are not syntax trivia; they explain what the backend remembers while it runs.

compileLanguageProfiles

compileLanguageProfiles is the compile workspace rulebook. Each language entry defines default filenames, file extensions, display labels, required tools, and optional Winget package ids. Compile, build, run, simulate, and install-tools operations all consult this object.

Server state excerpt: compileLanguageProfiles

```
const compileLanguageProfiles = {
  c: {
    defaultFile: "main.c",
    extensions: [".c", ".h"],
    label: "C",
    primaryTools: ["gcc"],
    winget: ["BrechtSanders.WinLibs.POSIX.UCRT"]
  },
  cpp: {
    defaultFile: "main.cpp",
    extensions: [".cpp", ".cc", ".cxx", ".hpp", ".h"],
    label: "C++",
    primaryTools: ["g++"],
    winget: ["BrechtSanders.WinLibs.POSIX.UCRT"]
  },
  verilog: {
    defaultFile: "design.v",
    extensions: [".v"],
    label: "Verilog",
    primaryTools: ["iverilog", "vvp"],
    winget: ["Icarus.Verilog"]
  },
  systemverilog: {
    defaultFile: "design.sv",
    extensions: [".sv", ".svh"],
    label: "SystemVerilog",
    primaryTools: ["iverilog", "vvp"],
    winget: ["Icarus.Verilog"]
  },
  ltspice: {
    defaultFile: "simulation.cir",
    extensions: [".cir", ".net", ".sp", ".asc"],
    label: "LTspice",
    primaryTools: ["ltspice"],
    winget: []
  },
  java: {
    defaultFile: "Main.java",
    extensions: [".java"],
    label: "Java",
    primaryTools: ["javac", "java"],
    winget: ["EclipseAdoptium.Temurin.21.JDK"]
  },
  javascript: {
    defaultFile: "main.js",
    extensions: [".js", ".mjs", ".cjs"],
    label: "JavaScript",
    primaryTools: ["node"],
    winget: []
  },
  python: {
    defaultFile: "main.py",
    extensions: [".py"],
```

```

    label: "Python",
    primaryTools: ["python"],
    winget: []
  },
  html: {
    defaultFile: "index.html",
    extensions: [".html", ".htm"],
    label: "HTML",
    primaryTools: [],
    winget: []
  }
};

```

compileToolCandidates

compileToolCandidates is the tool-location map. It gives findTool realistic places to search for gcc, g++, javac, java, node, python, iverilog, vvp, and LTspice before declaring a compiler missing.

Server state excerpt: compileToolCandidates

```

const compileToolCandidates = {
  gcc: [
    "gcc",
    process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Microsoft", "WinGet", "Packages",
    "BrechtSanders.WinLibs.POSIX.UCRT_Microsoft.Winget.Source_8wekyb3d8bbwe", "mingw64", "bin", "gcc.exe") : ""
  ],
  "g++": [
    "g++",
    "c++",
    process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Microsoft", "WinGet", "Packages",
    "BrechtSanders.WinLibs.POSIX.UCRT_Microsoft.Winget.Source_8wekyb3d8bbwe", "mingw64", "bin", "g++.exe") : ""
  ],
  clang: ["clang"],
  "clang++": ["clang++"],
  cl: ["cl"],
  javac: [
    "javac",
    "C:\\Program Files\\Eclipse Adoptium\\jdk-21.0.11.10-hotspot\\bin\\javac.exe"
  ],
  java: [
    "java",
    "C:\\Program Files\\Eclipse Adoptium\\jdk-21.0.11.10-hotspot\\bin\\java.exe"
  ],
  node: [
    /(?:^[\\\/])node(?:\.exe)?$/i.test(process.execPath || "") ? process.execPath : "",
    "node",
    "C:\\Program Files\\nodejs\\node.exe",
    process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Programs", "nodejs", "node.exe") : ""
  ],
  python: [process.env.PYTHON, "python", "py"],
  iverilog: ["iverilog", "C:\\iverilog\\bin\\iverilog.exe"],
  vvp: ["vvp", "C:\\iverilog\\bin\\vvp.exe"],
  ltspice: [
    process.env.LTSPICE_EXE,
    "C:\\Program Files\\ADI\\LTspice\\LTspice.exe",
    "C:\\Program Files\\LTC\\LTspiceXVII\\XVIIx64.exe",
    "C:\\Program Files\\Analog Devices\\LTspice\\LTspice.exe",
    process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Programs", "ADI", "LTspice", "LTspice.exe") : ""
  ]
};

```

Function walkthrough

normalizeCodeLanguage

normalizeCodeLanguage accepts loose human or UI language names and turns them into canonical backend ids. It trims the value, lowercases it, removes separator noise, and checks an alias map.

Full source for normalizeCodeLanguage

```

function normalizeCodeLanguage(value = "") {
  const clean = String(value || "").trim().toLowerCase().replace(/[_\s-]+/g, "");
  const aliases = {
    c: "c",
    cpp: "cpp",
    cplusplus: "cpp",

```

```

    "c++": "cpp",
    systemverilog: "systemverilog",
    sv: "systemverilog",
    verilog: "verilog",
    v: "verilog",
    ltspice: "ltspice",
    spice: "ltspice",
    cir: "ltspice",
    java: "java",
    javascript: "javascript",
    js: "javascript",
    node: "javascript",
    python: "python",
    py: "python",
    html: "html",
    htm: "html"
  };
  return aliases[clean] || (compileLanguageProfiles[clean] ? clean : "javascript");
}

```

The alias table is the center of the function. Values such as c++, cpp, sv, system verilog, py, js, node, spice, and cir collapse into the ids used by compileLanguageProfiles. If the cleaned value already names a known profile, the function keeps it. Otherwise JavaScript becomes the fallback.

This function prevents every compile branch from having to understand every spelling a user might type. The rest of the backend can switch on c, cpp, verilog, systemverilog, java, javascript, python, html, or ltspice.

findTool

findTool locates a compiler, runtime, simulator, or installer helper. It starts with cached answers so repeated compile runs do not rescan the machine. If no cache entry exists, it reads the candidate list for the requested tool name.

Full source for findTool

```

async function findTool(toolName) {
  if (compileToolCache.has(toolName)) return compileToolCache.get(toolName);
  const candidates = (compileToolCandidates[toolName] || [toolName]).filter(Boolean);
  for (const candidate of candidates) {
    if (path.isAbsolute(candidate)) {
      if (await pathExists(candidate)) {
        compileToolCache.set(toolName, candidate);
        return candidate;
      }
      continue;
    }
    try {
      const command = process.platform === "win32" ? "where" : "command";
      const args = process.platform === "win32" ? [candidate] : ["-v", candidate];
      const result = await execFileAsync(command, args, { timeout: 5000, windowsHide: true });
      const found = String(result.stdout || "").split(/\r?\n/).map((line) => line.trim()).find(Boolean);
      if (found) {
        compileToolCache.set(toolName, found);
        return found;
      }
    } catch {
      // Try the next candidate.
    }
  }

  if (["gcc", "g++"].includes(toolName) && process.env.LOCALAPPDATA) {
    const found = await findExecutableUnder(
      path.join(process.env.LOCALAPPDATA, "Microsoft", "WinGet", "Packages"),
      [toolName === "gcc" ? "gcc.exe" : "g++.exe"],
      7
    );
    compileToolCache.set(toolName, found || "");
    return found || "";
  }

  if (["javac", "java"].includes(toolName)) {
    const found = await findExecutableUnder("C:\\Program Files\\Eclipse Adoptium", [`${toolName}.exe`], 5);
    compileToolCache.set(toolName, found || "");
    return found || "";
  }
}

```

```

compileToolCache.set(toolName, "");
return "";
}

```

The function walks through realistic possibilities in order: explicit executable paths, commands available on PATH, and known installation folders. When it finds something usable, it stores that answer in compileToolCache and returns it. When it fails, it caches an empty string so the same missing tool does not cause repeated expensive searches.

This is why the builder can run on different Windows machines without hardcoding one install layout. The compiler feature asks for gcc or iverilog; findTool translates that request into an actual executable path if one exists.

runProcess

runProcess is the command execution wrapper. The backend uses it for compilers, simulators, Git, Java, Python, Node, and installer-related commands. The function begins by choosing a timeout and working directory, then starts the child process with shell disabled and the environment explicitly composed.

Full source for runProcess

```

async function runProcess(command, args = [], options = {}) {
  const timeoutMs = options.timeoutMs || 20000;
  const cwd = options.cwd || compileRoot;
  return new Promise((resolve) => {
    const startedAt = Date.now();
    const child = spawn(command, args, {
      cwd,
      env: { ...process.env, ...(options.env || {}) },
      shell: false,
      windowsHide: true
    });
    let stdout = "";
    let stderr = "";
    let timedOut = false;
    const timer = setTimeout(() => {
      timedOut = true;
      if (process.platform === "win32" && child.pid) {
        execFile("taskkill", ["/pid", String(child.pid), "/T", "/F"], { windowsHide: true }, () => {});
      }
      child.kill("SIGKILL");
    }, timeoutMs);
    child.stdout?.on("data", (chunk) => {
      stdout += chunk.toString();
    });
    child.stderr?.on("data", (chunk) => {
      stderr += chunk.toString();
    });
    if (Object.prototype.hasOwnProperty.call(options, "input")) {
      child.stdin?.write(String(options.input || ""));
    }
    child.stdin?.end();
    child.on("error", (error) => {
      clearTimeout(timer);
      resolve({ ok: false, code: -1, stdout, stderr: `${stderr}\n${error.message}`.trim(), timedOut, elapsedMs:
Date.now() - startedAt });
    });
    child.on("close", (code) => {
      clearTimeout(timer);
      resolve({ ok: !timedOut && code === 0, code, stdout, stderr, timedOut, elapsedMs: Date.now() - startedAt });
    });
  });
}

```

After spawn, the function becomes a recorder. It accumulates stdout and stderr as data arrives, writes stdin if the caller supplied input, and ends stdin so tools waiting for input do not hang forever. A timer watches the process. On timeout, Windows process trees are killed through taskkill before the child receives SIGKILL.

The result object is the value of the wrapper. Instead of making every caller interpret Node child-process events, the function returns ok, code, stdout, stderr, timedOut, and elapsedMs. That shape is why Compile Code and Git publishing can show terminal output consistently.

compileWorkspaceFilesFromPayload

`compileWorkspaceFilesFromPayload` turns frontend editor state into a clean list of files for a project workspace. It starts with `payload.workspaceFiles`, then defines an inner `addFile` helper that normalizes each file before placing it in a `Map`.

Full source for compileWorkspaceFilesFromPayload

```
function compileWorkspaceFilesFromPayload(payload = {}, activeFileName = "", activeLanguage = "javascript") {
  const incoming = Array.isArray(payload.workspaceFiles) ? payload.workspaceFiles : [];
  const byId = new Map();
  const addFile = (item = {}, fallbackId = "") => {
    const code = String(item.code || "");
    if (!code.trim()) return;
    const language = normalizeCodeLanguage(item.language || detectCodeLanguageFromSource(code, item.fileName));
    const fileName = safeCodeFileName(item.fileName || activeFileName ||
    compileLanguageProfiles[language]?.defaultFile || "source.txt", language);
    const id = safeSegment(item.id || fallbackId || fileName, path.parse(fileName).name);
    byId.set(id, {
      id,
      title: clampText(item.title || fileName, 180),
      fileName,
      language,
      role: normalizeCompileFileRole(item.role || inferCompileFileRole(fileName, code, language), language),
      code
    });
  };
  incoming.forEach((item, index) => addFile(item, `workspace-${index + 1}`));
  addFile({
    id: payload.fileId,
    title: payload.title,
    fileName: activeFileName,
    language: activeLanguage,
    role: payload.role,
    code: payload.code
  }, "active");
  return [...byId.values()].sort((a, b) => a.fileName.localeCompare(b.fileName));
}
```

The helper ignores empty code, detects or normalizes the language, chooses a safe filename, creates a safe id, normalizes HDL roles, and stores the result by id. After processing the incoming workspace files, it adds the active file as well. That last step matters because the editor's active file might not already be present in the workspace list.

The returned array is sorted by filename. Compile Code can then write and compile a stable set of files rather than trying to reason about raw frontend payloads.

writeCompileWorkspaceSources

`writeCompileWorkspaceSources` is where normalized workspace objects become real files. It creates the target directory, tracks duplicate filenames, filters by language or extension when requested, writes each source file, and returns records that include `uniqueName` and `sourcePath`.

Full source for writeCompileWorkspaceSources

```
async function writeCompileWorkspaceSources(files = [], targetDir = "", options = {}) {
  await mkdir(targetDir, { recursive: true });
  const seen = new Map();
  const written = [];
  for (const file of files) {
    if (options.languages?.length && !options.languages.includes(normalizeCodeLanguage(file.language))) continue;
    if (options.extensions?.length && !options.extensions.includes(path.extname(file.fileName).toLowerCase()))
    continue;
    const parsed = path.parse(file.fileName);
    const count = seen.get(file.fileName) || 0;
    seen.set(file.fileName, count + 1);
    const uniqueName = count ? `${parsed.name}_${count}${parsed.ext}` : file.fileName;
    const sourcePath = path.join(targetDir, uniqueName);
    await writeFile(sourcePath, file.code, "utf8");
    written.push({ ...file, uniqueName, sourcePath });
  }
  return written;
}
```

Duplicate handling is the most practical part of the body. If two workspace entries claim the same filename, the second one receives a suffix before its extension. That avoids overwriting one source with another while still preserving a readable name in compiler output.

Compilers do not compile rich editor blocks. They compile files. This function is the bridge between project editor state and the physical source files a toolchain can consume.

parseVcdScopeText

parseVcdScopeText converts a Verilog VCD waveform file into data a scope viewer can draw. It reads the text line by line and keeps separate state for scopes, signal definitions, timescale text, current time, and the largest observed time.

Full source for parseVcdScopeText

```
function parseVcdScopeText(text = "", source = "waveform.vcd") {
  const lines = String(text || "").split(/\r?\n/);
  const scopes = [];
  const signalsByCode = new Map();
  const timeScaleParts = [];
  let inTimescale = false;
  let time = 0;
  let maxTime = 0;
  let definitionDone = false;
  for (const rawLine of lines) {
    const line = rawLine.trim();
    if (!line) continue;
    if (line.startsWith("$timescale")) {
      inTimescale = true;
      const inline = line.replace("$timescale", "").replace("$end", "").trim();
      if (inline) timeScaleParts.push(inline);
      continue;
    }
    if (inTimescale) {
      if (line === "$end") {
        inTimescale = false;
      } else {
        timeScaleParts.push(line.replace("$end", "").trim());
        if (line.includes("$end")) inTimescale = false;
      }
      continue;
    }
    const scopeMatch = line.match(/^$scope\s+\S+\s+(.+) \s+\$end$/);
    if (scopeMatch) {
      scopes.push(scopeMatch[1]);
      continue;
    }
    if (/^$upscope\b/.test(line)) {
      scopes.pop();
      continue;
    }
    const varMatch = line.match(/^$var\s+\S+\s+(\d+)\s+(\S+)\s+(.+) \s+\$end$/);
    if (varMatch) {
      const [, width, code, reference] = varMatch;
      if (!signalsByCode.has(code) && signalsByCode.size < 64) {
        const cleanReference = reference.replace(/\s+\[^\]]+\]/, "");
        signalsByCode.set(code, {
          code,
          name: [...scopes, cleanReference].filter(Boolean).join("."),
          reference: cleanReference,
          width: Number(width) || 1,
          changes: []
        });
      }
      continue;
    }
    if (line.startsWith("$enddefinitions")) {
      definitionDone = true;
      continue;
    }
    if (!definitionDone) continue;
    if (line.startsWith("#")) {
      time = Number(line.slice(1)) || 0;
      maxTime = Math.max(maxTime, time);
      continue;
    }
  }
}
```

```

    }
    let code = "";
    let value = "";
    const vectorMatch = line.match(/^(?  
[01xz_]+)\s+(\S+)\$/i);
    if (vectorMatch) {
        value = normalizeVcdValue(vectorMatch[1]);
        code = vectorMatch[2];
    } else if (/^[01xz]/i.test(line)) {
        value = normalizeVcdValue(line[0]);
        code = line.slice(1).trim();
    }
    const signal = signalsByCode.get(code);
    if (!signal || !value) continue;
    const previous = signal.changes[signal.changes.length - 1];
    if (!previous || previous.value !== value || previous.time !== time) {
        if (signal.changes.length < 600) signal.changes.push({ time, value });
    }
}
const signals = [...signalsByCode.values()]
    .filter((signal) => signal.changes.length)
    .slice(0, 32);
return {
    source: path.basename(source),
    timeScale: timeScaleParts.join(" ").replace(/\\s+/g, " ").trim() || "ticks",
    maxTime,
    signals
};
}

```

The first part of the loop handles metadata. Timescale lines are collected until their end marker. Scope lines push and pop hierarchical names. \$var lines register signal codes, widths, and references, but the function caps the number of signals so a huge waveform does not overwhelm the UI.

After \$enddefinitions, the function switches from definitions to signal changes. Lines beginning with # update the current simulation time. Scalar and vector value-change lines are normalized, matched to their signal code, and appended only when the value actually changes. The returned object contains the waveform source name, timescale, max time, and a trimmed signal list.

compileAndRunCode

compileAndRunCode is the backend IDE engine. It starts by deciding the language from the requested language or detected source. Then it chooses the language profile, produces a safe active filename, saves the active source, builds a normalized workspace list, and prepares local helpers for run directories, terminal text, and missing-tool checks.

Full source for compileAndRunCode

```

async function compileAndRunCode(payload = {}) {
    const requestedLanguage = String(payload.language || "").trim().toLowerCase();
    const language = requestedLanguage && requestedLanguage !== "auto"
        ? normalizeCodeLanguage(requestedLanguage)
        : detectCodeLanguageFromSource(payload.code, payload.fileName);
    const profile = compileLanguageProfiles[language] || compileLanguageProfiles.javascript;
    const fileName = safeCodeFileName(payload.fileName, language);
    const saved = await saveCompileSource({ ...payload, language, fileName });
    const projectFolder = safeSegment(payload.projectId, "project");
    const sourcePath = resolveInsideCompileRoot(projectFolder, safeSegment(saved.id), saved.fileName);
    const sourceCode = String(payload.code || "");
    const action = compileActionFromPayload(payload.action, language);
    const allWorkspaceFiles = compileWorkspaceFilesFromPayload(payload, saved.fileName, language);
    let runDir = "";
    let runSourcePath = "";
    let runWorkspaceSources = [];
    const ensureRunSource = async (options = {}) => {
        if (runDir && runSourcePath) return { runDir, runSourcePath, writtenSources: runWorkspaceSources };
        const runId = `${Date.now()}-${Math.random().toString(16).slice(2)}`;
        runDir = resolveInsideCompileRoot(projectFolder, ".runs", runId);
        await mkdir(runDir, { recursive: true });
        const filesToWrite = Array.isArray(options.files) && options.files.length ? options.files : allWorkspaceFiles;
        runWorkspaceSources = await writeCompileWorkspaceSources(filesToWrite, runDir, {
            languages: options.languages,
            extensions: options.extensions
        });
    };
    const active = runWorkspaceSources.find((file) => file.id === saved.id || file.fileName === saved.fileName);
    runSourcePath = active?.sourcePath || path.join(runDir, saved.fileName);
}

```

```

if (!active) {
  await cp(sourcePath, runSourcePath, { force: true });
  runWorkspaceSources.push({
    id: saved.id,
    title: saved.title,
    fileName: saved.fileName,
    language,
    role: saved.role,
    code: sourceCode,
    uniqueName: saved.fileName,
    sourcePath: runSourcePath
  });
}
return { runDir, runSourcePath, writtenSources: runWorkspaceSources };
};
const stdin = String(payload.stdin || "");
const forceRebuild = payload.forceRebuild === true;
const terminal = [];
terminal.push(`${compileActionLabel(action, language)} request for ${saved.fileName}`);
const append = (label, result) => {
  terminal.push(terminalLine(label, processTerminalText(result)));
};
const missing = async (tools) => {
  const found = {};
  for (const tool of tools) found[tool] = await findTool(tool);
  const missingTools = tools.filter((tool) => !found[tool]);
  return { found, missingTools };
};

if (language === "html") {
  const openTags = (String(payload.code || "").match(/<([a-z][\w:-]*)\b[^>]*>/gi) || []).length;
  const closeTags = (String(payload.code || "").match(/<\/([a-z][\w:-]*)>/gi) || []).length;
  const ok = openTags >= closeTags;
  return {
    ok,
    language,
    saved,
    terminal: [
      "HTML validation pass",
      ok ? "No obvious tag-balance issue was detected." : "The file may have more closing tags than opening tags.",
      `Saved source: ${saved.sourcePath}`
    ].join("\n")
  };
}

const stdinOptions = stdin ? { input: stdin } : {};
let ok = false;

if (language === "javascript") {
  const node = await findTool("node");
  if (!node) return { ok: false, language, saved, terminal: "Node.js was not found. Install Node.js to run JavaScript." };
  const jsFiles = allWorkspaceFiles.filter((file) => normalizeCodeLanguage(file.language) === "javascript");
  const runSource = await ensureRunSource({ files: jsFiles.length ? jsFiles : allWorkspaceFiles, languages: ["javascript"] });
  const targets = action === "build"
    ? runSource.writtenSources.filter((file) => normalizeCodeLanguage(file.language) === "javascript")
    : runSource.writtenSources.filter((file) => file.sourcePath === runSource.runSourcePath);
  ok = true;
  for (const target of targets.length ? targets : [{ sourcePath: runSource.runSourcePath, uniqueName: saved.fileName }]) {
    const check = await runProcess(node, ["--check", target.sourcePath], { cwd: runSource.runDir, timeoutMs: 15000 });
    terminal.push(terminalLine(`${path.basename(node)} --check ${sourceDisplayName(target)}`, processTerminalText(check)));
    ok = ok && check.ok;
  }
  if (action === "compile" || action === "build") {
    terminal.push(ok ? "JavaScript syntax check completed." : "JavaScript syntax check found errors.");
  } else if (ok) {
    const run = await runProcess(node, [runSource.runSourcePath], { cwd: runSource.runDir, timeoutMs: 20000, ...stdinOptions });
    append(`${path.basename(node)} ${saved.fileName}`, run);
    ok = run.ok;
  }
} else if (language === "python") {

```

```

const python = await findTool("python");
if (!python) return { ok: false, language, saved, terminal: "Python was not found. Install Python to run Python code." };
const pyFiles = allWorkspaceFiles.filter((file) => normalizeCodeLanguage(file.language) === "python");
const runSource = await ensureRunSource({ files: pyFiles.length ? pyFiles : allWorkspaceFiles, languages: ["python"] });
const targets = action === "build"
  ? runSource.writtenSources.filter((file) => normalizeCodeLanguage(file.language) === "python")
  : runSource.writtenSources.filter((file) => file.sourcePath === runSource.runSourcePath);
ok = true;
for (const target of targets.length ? targets : [{ sourcePath: runSource.runSourcePath, uniqueName: saved.fileName }]) {
  const check = await runProcess(python, ["-m", "py_compile", target.sourcePath], { cwd: runSource.runDir, timeoutMs: 15000 });
  terminal.push(terminalLine(`${path.basename(python)} -m py_compile ${sourceDisplayName(target)}`, processTerminalText(check)));
  ok = ok && check.ok;
}
if (action === "compile" || action === "build") {
  terminal.push(ok ? "Python bytecode compile check completed." : "Python compile check found errors.");
} else if (ok) {
  const run = await runProcess(python, [runSource.runSourcePath], { cwd: runSource.runDir, timeoutMs: 20000, ...stdinOptions });
  append(`${path.basename(python)} ${saved.fileName}`, run);
  ok = run.ok;
}
} else if (language === "c" || language === "cpp") {
  const cProfile = cFamilyCompileProfile(language, saved.fileName);
  const toolName = cProfile.compiler;
  const { found, missingTools } = await missing([toolName]);
  if (missingTools.length) {
    return { ok: false, language, saved, terminal: `${profile.label} compiler missing: ${missingTools.join(", ")}. Install WinLibs/MinGW or add it to PATH.` };
  }
  const workspaceSources = cFamilyWorkspaceSources(allWorkspaceFiles, language, saved.fileName);
  const buildTargets = action === "build"
    ? workspaceSources.filter((file) => !cFamilyCompileProfile(language, file.fileName).header)
    : workspaceSources.filter((file) => file.fileName === saved.fileName);
  const compilerVersion = await toolVersionLine(found[toolName]);
  const cacheKey = compileCacheKey({
    language,
    action,
    activeFileName: saved.fileName,
    files: buildTargets.map((file) => ({ fileName: file.fileName, code: file.code })),
    compiler: found[toolName],
    compilerVersion,
    flags: cProfile.flags,
    header: cProfile.header
  });
  const cacheDir = compileCacheDirectory(payload.projectId, language, cacheKey);
  await mkdir(cacheDir, { recursive: true });
  const workspaceDir = path.join(cacheDir, "sources");
  await rm(workspaceDir, { recursive: true, force: true });
  const writtenSources = await writeCompileWorkspaceSources(workspaceSources, workspaceDir);
  const activeWritten = writtenSources.find((file) => file.fileName === saved.fileName) || writtenSources[0];
  const binaryName = cFamilyBinaryName(saved.fileName);
  const output = path.join(cacheDir, binaryName);
  const outputMarker = path.join(cacheDir, "syntax-ok.txt");
  const cached = !forceRebuild && await pathExists(output);
  const cachedHeader = cProfile.header && !forceRebuild && await pathExists(outputMarker);
  terminal.push(`${cProfile.label} build profile: ${cProfile.standard}, warnings enabled, debug symbols enabled`);
  terminal.push(`Workspace sources visible to compiler: ${writtenSources.map(sourceDisplayName).join(", ")}`);
  saved.fileName});
  if (compilerVersion) terminal.push(`Compiler: ${compilerVersion}`);
  if (cached || cachedHeader) {
    terminal.push(cachedBuildLine(cProfile.header ? `${cProfile.label} header syntax check` : `${cProfile.label} binary`, cProfile.header ? outputMarker : output));
  } else {
    const args = cProfile.header
      ? ["-x", cProfile.languageFlag, "-fsyntax-only", ...cProfile.flags, activeWritten?.sourcePath || sourcePath]
      : [
        ...(action === "build"
          ? writtenSources.filter((file) => !cFamilyCompileProfile(language, file.fileName).header).map((file) => file.sourcePath)
          : [activeWritten?.sourcePath || sourcePath]),
        "-o",
        output,
        ...cProfile.flags
      ];

```

```

    ];
    const compile = await runProcess(found[toolName], args, { cwd: workspaceDir, timeoutMs: 30000 });
    const replacements = [
      { from: sourcePath, to: saved.fileName },
      { from: output, to: binaryName },
      ...writtenSources.map((file) => ({ from: file.sourcePath, to: sourceDisplayName(file) })))
    ];
    terminal.push(terminalLine(
      `${path.basename(found[toolName])} ${action === "build" ? "workspace sources" : saved.fileName} $
{cProfile.header ? "-fsyntax-only" : "-o ${binaryName}`} ${cProfile.flags.join(" ")}`,
      processTerminalTextWithPaths(compile, replacements)
    ));
    ok = compile.ok;
    if (ok && cProfile.header) {
      await writeFile(outputMarker, `${cProfile.label} header syntax check passed for ${saved.fileName}\n`,
"utf8");
    }
  }
  if (cProfile.header && (cachedHeader || ok)) {
    terminal.push(`Header syntax check passed: ${saved.fileName}`);
    ok = true;
  } else if (cached || ok) {
    terminal.push(`Generated binary: ${output}`);
    if (action === "run") {
      const toolPath = found[toolName];
      const run = await runProcess(output, [], {
        cwd: cacheDir,
        timeoutMs: 20000,
        env: { PATH: `${path.dirname(toolPath)}${path.delimiter}${process.env.PATH || ""}` },
        ...stdinOptions
      });
      terminal.push(terminalLine(binaryName, cFamilyRunOutput(run)));
      ok = run.ok;
    } else {
      terminal.push(`${compileActionLabel(action, language)} completed. Use Run to execute the generated
binary.`);
      ok = true;
    }
  }
} else if (language === "java") {
  const { found, missingTools } = await missing(["javac", "java"]);
  if (missingTools.length) {
    return { ok: false, language, saved, terminal: `Java tools missing: ${missingTools.join(", ")}. Install a JDK
or add it to PATH.` };
  }
  const className = javaMainClassName(payload.code, saved.fileName);
  const javaFiles = allWorkspaceFiles.filter((file) => normalizeCodeLanguage(file.language) === "java");
  const cacheKey = compileCacheKey({
    language,
    fileName: saved.fileName,
    files: javaFiles.map((file) => ({ fileName: file.fileName, code: file.code })),
    compiler: found.javac,
    runtime: found.java,
    className
  });
  const cacheDir = compileCacheDirectory(payload.projectId, language, cacheKey);
  await mkdir(cacheDir, { recursive: true });
  const classPath = path.join(cacheDir, `${className}.class`);
  const sourceDir = path.join(cacheDir, "sources");
  await rm(sourceDir, { recursive: true, force: true });
  const writtenSources = await writeCompileWorkspaceSources(javaFiles.length ? javaFiles : allWorkspaceFiles,
sourceDir, { languages: ["java"] });
  const javaPath = writtenSources.find((file) => file.fileName === saved.fileName)?.sourcePath ||
path.join(sourceDir, `${className}.java`);
  const cached = !forceRebuild && await pathExists(classPath);
  terminal.push(`Java workspace sources: ${writtenSources.map(sourceDisplayName).join(", ") || saved.fileName}`);
  if (cached) {
    terminal.push(cachedBuildLine("Java class", classPath));
  } else {
    if (!writtenSources.length) await writeFile(javaPath, sourceCode, "utf8");
    const compileTargets = action === "build" ? writtenSources.map((file) => file.sourcePath) : [javaPath];
    const compile = await runProcess(found.javac, ["-d", cacheDir, ...compileTargets], { cwd: sourceDir,
timeoutMs: 30000 });
    terminal.push(terminalLine(`${path.basename(found.javac)} ${action === "build" ? "workspace sources" :
path.basename(javaPath)}`, processTerminalTextWithPaths(
      compile,
      writtenSources.map((file) => ({ from: file.sourcePath, to: sourceDisplayName(file) })))
    ));
  }
}

```

```

    ok = compile.ok;
  }
  if ((cached || ok) && action !== "compile" && action !== "build") {
    terminal.push(`Generated class: ${className}`);
    const run = await runProcess(found.java, ["-cp", cacheDir, className], { cwd: cacheDir, timeoutMs: 20000,
...stdinOptions });
    append(`${path.basename(found.java)} ${className}`, run);
    ok = run.ok;
  } else if (cached || ok) {
    terminal.push(`${compileActionLabel(action, language)} completed. Use Run to execute ${className}.`);
    ok = true;
  }
} else if (language === "verilog" || language === "systemverilog") {
  const { found, missingTools } = await missing(["iverilog", "vvp"]);
  if (missingTools.length) {
    return { ok: false, language, saved, terminal: `${profile.label} simulator tools missing: $
{missingTools.join(", ")}. Install Icarus Verilog and add iverilog/vvp to PATH.` };
  }
  const hdlFiles = hdlFilesFromPayload(payload, saved.fileName, language);
  const testbenchFiles = hdlFiles.filter((file) => file.role === "testbench");
  if (action === "simulate" && !testbenchFiles.length) {
    return {
      ok: false,
      language,
      saved,
      terminal: [
        `${profile.label} simulation requires a testbench.`,
        "Mark one HDL file as Testbench or click Add testbench, connect it to the design under test, then run
again.",
        "A design-only file can be saved and appended to the portfolio, but simulation needs stimulus from a
testbench."
      ].join("\n")
    };
  }
  if (action === "simulate" && !testbenchFiles.some((file) => hdlHasWaveDump(file.code))) {
    return {
      ok: false,
      language,
      saved,
      terminal: [
        "HDL scope requires waveform dump calls in the testbench.",
        'Add $dumpfile("waveform.vcd"); and $dumpvars(0, tb_module_name); inside an initial block.',
        "The simulator will not run without a waveform dump because there would be no signal-over-time data for
the scope."
      ].join("\n")
    };
  }
}
const usesSystemVerilog = hdlFiles.some((file) => normalizeCodeLanguage(file.language) === "systemverilog");
const flags = usesSystemVerilog ? ["-g2012", "-Wall"] : ["-g2005-sv", "-Wall"];
const topModule = testbenchFiles.flatMap((file) => hdlModuleNames(file.code)).find(Boolean) || "";
const cacheKey = compileCacheKey({
  language: usesSystemVerilog ? "systemverilog" : "verilog",
  files: hdlFiles.map((file) => ({
    fileName: file.fileName,
    language: file.language,
    role: file.role,
    code: file.code
  })),
  compiler: found.iverilog,
  runtime: found.vvp,
  flags,
  topModule
});
const cacheDir = compileCacheDirectory(payload.projectId, language, cacheKey);
await mkdir(cacheDir, { recursive: true });
const output = path.join(cacheDir, "simulation.vvp");
const sourceFiles = await writeHdlSimulationSources(hdlFiles, cacheDir);
const designCount = sourceFiles.filter((file) => file.role !== "testbench").length;
terminal.push(`${profile.label} simulation set: ${designCount} design file${designCount === 1 ? "" : "s"}, $
{testbenchFiles.length} testbench file${testbenchFiles.length === 1 ? "" : "s"}`);
if (topModule) terminal.push(`Testbench top: ${topModule}`);
const cached = !forceRebuild && await pathExists(output);
if (cached) {
  terminal.push(cachedBuildLine(`${profile.label} simulation`, output));
} else {
  const compileArgs = [
    ...flags,
    ...(topModule ? ["-s", topModule] : []),

```

```

        "-o",
        output,
        ...sourceFiles.map((file) => file.sourcePath)
    ];
    const compile = await runProcess(found.iverilog, compileArgs, { cwd: cacheDir, timeoutMs: 30000 });
    const replacements = sourceFiles.map((file) => ({ from: file.sourcePath, to: file.uniqueName }));
    terminal.push(terminalLine(
        `${path.basename(found.iverilog)} ${flags.join(" ")}${topModule ? ` -s ${topModule}` : ""} -o
simulation.vvp`,
        processTerminalTextWithPaths(compile, replacements)
    ));
    ok = compile.ok;
}
let waveform = null;
if (cached || ok) {
    terminal.push(`Generated simulation: ${output}`);
    if (action === "compile" || action === "build") {
        terminal.push(`${compileActionLabel(action, language)} completed. Use Simulate to run the generated
simulation and produce scope data.`);
        return {
            ok: true,
            language,
            saved,
            waveform: null,
            terminal: terminal.join("\n\n").trim() || "No compiler output was returned."
        };
    }
    await clearHdlWaveforms(cacheDir);
    const run = await runProcess(found.vvp, [output], { cwd: cacheDir, timeoutMs: 20000 });
    terminal.push(terminalLine(`${path.basename(found.vvp)} simulation.vvp`, cleanHdlSimulationOutput(run)));
    waveform = run.ok ? await readHdlWaveform(cacheDir) : null;
    if (waveform?.signals?.length) {
        terminal.push(`Scope waveform: ${waveform.source} (${waveform.signals.length} signals, 0 to $
{waveform.maxTime} ${waveform.timeScale})`);
    } else {
        terminal.push("Scope waveform was not generated. Confirm the testbench executes $dumpfile and $dumpvars
before $finish.");
    }
    ok = run.ok && Boolean(waveform?.signals?.length);
    return {
        ok,
        language,
        saved,
        waveform,
        terminal: terminal.join("\n\n").trim() || "No compiler output was returned."
    };
}
} else if (language === "ltspice") {
    const ltspice = await findTool("ltspice");
    if (!ltspice) {
        return { ok: false, language, saved, terminal: "LTspice executable was not found. Install LTspice or set
LTSPICE_EXE to the executable path." };
    }
    const runSource = await ensureRunSource();
    const run = await runProcess(ltspice, ["-b", runSource.runSourcePath], { cwd: runSource.runDir, timeoutMs: 45000
});
    append(`${path.basename(ltspice)} -b ${saved.fileName}`, run);
    const logPath = runSource.runSourcePath.replace(/\.([^.]+)$/, ".log");
    if (await pathExists(logPath)) terminal.push(await readFile(logPath, "utf8"));
    ok = run.ok;
}

return {
    ok,
    language,
    saved,
    terminal: terminal.join("\n\n").trim() || "No compiler output was returned."
};
}

```

The body then becomes a language dispatcher. HTML receives a lightweight tag-balance validation. JavaScript and Python perform syntax checks first, then run the active file when the requested action is run. C and C++ select a compiler profile, write workspace sources, compute a cache key, compile with warnings and debug flags, and run the generated binary only when requested. Java compiles into a cache directory and runs the detected class name. Verilog and SystemVerilog require a testbench and waveform dump before simulation,

compile sources through iverilog, run vvp, and parse VCD output for the scope. LTspice runs the simulator in batch mode and includes the log file if one appears.

The large size of this function is intentional because it is the point where one frontend command becomes several very different toolchain behaviors. The function keeps the UI simple: the frontend sends one payload with action, code, language, files, project id, and stdin; the backend returns saved metadata, terminal text, ok status, and waveform data when HDL simulation succeeds.

installCompilerTools

installCompilerTools is the automatic setup path for language toolchains that have Winget packages configured. It normalizes the selected language, reads the language profile, and stops early with a clear message when no automatic install rule exists.

Full source for installCompilerTools

```
async function installCompilerTools(language = "") {
  const lang = normalizeCodeLanguage(language);
  const profile = compileLanguageProfiles[lang] || compileLanguageProfiles.javascript;
  if (!profile.winget?.length) {
    return {
      ok: false,
      language: lang,
      terminal: `${profile.label} does not have an automatic Winget compiler install configured. Install the
required tools manually, then restart the builder.`
    };
  }
  const winget = await findTool("winget") || "winget";
  const terminal = [];
  let ok = true;
  for (const packageId of profile.winget) {
    const args = [
      "install",
      "--source",
      "winget",
      "--accept-source-agreements",
      "--accept-package-agreements",
      "--silent",
      "--id",
      packageId,
      "--exact"
    ];
    const result = await runProcess(winget, args, { cwd: root, timeoutMs: 600000 });
    terminal.push(terminalLine(`winget ${args.join(" ")}`), processTerminalText(result));
    ok = ok && result.ok;
  }
  compileToolCache.clear();
  return { ok, language: lang, terminal: terminal.join("\n\n"), tools: await compileToolStatus() };
}
```

When a Winget list exists, the function runs each package install with source and package agreements accepted. It appends each command result to a terminal array and returns ok only if every package command succeeds. This gives the frontend a conventional installer-like log instead of a vague success or failure.

Chapter 5: Publishing, Authentication, And Loading From Target

Publishing is where local work becomes public website content. The functions in this chapter guard repository setup, daily authorization caching, target import, publish-mirror synchronization, and final Git push.

Important working state

These objects and constants are shown before the functions because they are the shared state the functions read from or protect. They are not syntax trivia; they explain what the backend remembers while it runs.

publishAuthCachePath

publishAuthCachePath stores the short-lived authorization record. The backend uses it to remember that the current machine, repository, branch, and workspace were recently verified instead of asking GitHub every time the app opens.

Server state excerpt: publishAuthCachePath

```
const publishAuthCachePath = path.join(root, ".omb-publish-session.json");
```

publishAuthCacheTtlMs

publishAuthCacheTtlMs expresses the normal one-day authorization window requested for publishing. It keeps the app convenient while still forcing periodic verification.

Server state excerpt: publishAuthCacheTtlMs

```
const publishAuthCacheTtlMs = 24 * 60 * 60 * 1000;
```

publishAuthExtendedThreshold

publishAuthExtendedThreshold is the rule that turns repeated successful authorizations into longer trust. In this app, repeated success over the recent window can stretch trust to thirty days.

Server state excerpt: publishAuthExtendedThreshold

```
const publishAuthExtendedThreshold = 3;
```

gitCandidates

gitCandidates is the Git discovery list. The backend tries environment paths, normal Program Files locations, and per-user Git installs because Windows machines do not all put Git in the same place.

Server state excerpt: gitCandidates

```
const gitCandidates = [
  process.env.GIT_EXE,
  "git",
  "C:\\Program Files\\Git\\cmd\\git.exe",
  "C:\\Program Files\\Git\\bin\\git.exe",
  "C:\\Program Files (x86)\\Git\\cmd\\git.exe",
  "C:\\Program Files (x86)\\Git\\bin\\git.exe",
  process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Programs", "Git", "cmd", "git.exe") : "",
  process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Programs", "Git", "bin", "git.exe") : ""
].filter(Boolean);
```

Function walkthrough

validatePublishRemoteUrl

validatePublishRemoteUrl turns a typed repository target into either a clean remote URL or a clear error. It accepts blank input when the caller is not setting a remote, but when a value exists it must match a GitHub HTTPS URL, SSH URL, or a plausible .git remote.

Full source for validatePublishRemoteUrl

```
function validatePublishRemoteUrl(value = "") {
  const remoteUrl = String(value || "").trim();
  if (!remoteUrl) return "";
  if (/^git@github\.com:[A-Za-z0-9_-]+\.[A-Za-z0-9_-]+(?:\.git)?$/i.test(remoteUrl)) return remoteUrl;
  try {
```

```

const parsed = new URL(remoteUrl);
if (parsed.protocol !== "https:" || parsed.hostname.toLowerCase() !== "github.com") {
  throw new Error("Only GitHub HTTPS repository URLs are supported by the guided setup.");
}
const parts = parsed.pathname.replace(/^\/+/, "").split("/");
if (parts.length < 2 || !parts[0] || !parts[1]) {
  throw new Error("GitHub repository URL must include owner and repository name.");
}
return `https://github.com/${parts[0]}/${parts[1].replace(/\.git$/i, "")}.git`;
} catch (error) {
  if (error.message?.includes("Only GitHub") || error.message?.includes("owner")) throw error;
  throw new Error("Enter a GitHub repository URL such as https://github.com/USERNAME/USERNAME.github.io.git.");
}
}

```

The function is deliberately early in the publish flow. A malformed repository URL should fail before Git remotes are changed, credentials are stored, or files are copied.

getPublishTargetInfo

getPublishTargetInfo asks the local publish workspace what repository it is connected to. It checks whether the folder is a Git worktree, reads the origin remote and current branch when available, parses GitHub owner/repo names, and checks for a CNAME file.

Full source for getPublishTargetInfo

```

async function getPublishTargetInfo() {
  const info = {
    workspace: root,
    portfolioWorkspace: portfolioRoot,
    separatedWorkspace: !samePath(root, portfolioRoot),
    defaultRepository: defaultSiteRepository,
    gitBacked: false,
    compatible: await workspaceHasCompatibleSiteFiles(),
    remote: "",
    branch: "",
    repository: "",
    customDomain: ""
  };

  try {
    info.gitBacked = (await runPublishGit(["rev-parse", "--is-inside-work-tree"])).stdout.trim() === "true";
  } catch {
    return info;
  }

  try {
    info.remote = remoteUrlForDisplay((await runPublishGit(["remote", "get-url", "origin"])).stdout.trim());
  } catch {
    info.remote = "";
  }

  try {
    info.branch = (await runPublishGit(["branch", "--show-current"])).stdout.trim();
  } catch {
    info.branch = "";
  }

  const parsedRemote = parseGitHubRemote(info.remote);
  info.repository = parsedRemote ? `${parsedRemote.owner}/${parsedRemote.repo}` : info.remote;

  try {
    info.customDomain = (await readFile(resolveInsidePortfolioRoot("CNAME"), "utf8")).trim();
  } catch {
    info.customDomain = "";
  }

  const accessShape = {
    branch: info.branch,
    remote: info.remote,
    repository: info.repository
  };
  const cached = await readPublishAuthCache();
  if (publishAuthCacheIsFresh(cached, accessShape)) {
    info.authorizationChecked = true;
  }
}

```

```

    info.authorizationCached = true;
    info.checkedAt = cached.checkedAt;
    info.expiresAt = publishAuthCacheExpiresAt(cached);
    info.extendedTrust = Boolean(cached.extendedTrust);
    info.successCountLastWeek = cached.successCountLastWeek || 0;
    info.trustDays = cached.trustDays || (cached.extendedTrust ? 30 : 1);
  } else {
    info.authorizationChecked = false;
    info.authorizationCached = false;
    info.checkedAt = "";
    info.expiresAt = "";
    info.extendedTrust = false;
    info.successCountLastWeek = cached?.successCountLastWeek || 0;
    info.trustDays = 0;
  }
}

return info;
}

```

The result is a status object rather than a thrown error in normal missing-target cases. That lets the builder show a useful target panel even when no repository has been associated yet.

configurePublishTarget

configurePublishTarget is the lower-level target setup helper. It reads repositoryUrl, customDomain, authUsername, and authPassword from the options object, validates each value, ensures the portfolio folder is a Git repository, and sets the origin remote when a remote was supplied.

Full source for configurePublishTarget

```

async function configurePublishTarget(options = {}) {
  const {
    repositoryUrl = "",
    customDomain = "",
    authUsername = "",
    authPassword = ""
  } = options;
  const customDomainProvided = Object.prototype.hasOwnProperty.call(options, "customDomain");
  const remoteUrl = validatePublishRemoteUrl(repositoryUrl);
  const domain = validateCustomDomain(customDomain);
  validateCredentialPair(authUsername, authPassword);

  await ensureGitRepository();

  if (remoteUrl) {
    await setOriginRemote(remoteUrl);
    const defaultBranch = await detectRemoteDefaultBranch(remoteUrl);
    if (defaultBranch) await checkoutPublishBranch(defaultBranch);
  }

  await writeTargetCustomDomain(domain, customDomainProvided);

  const currentRemote = remoteUrl || (await getPublishTargetInfo()).remote;
  const credentials = await storeGitCredentials(currentRemote, authUsername, authPassword);
  const target = await getPublishTargetInfo();
  return {
    ...target,
    credentialsStored: credentials.stored,
    credentialUsername: credentials.username || ""
  };
}

```

After the remote is set, the function detects the remote default branch and checks out that branch locally. Then it writes or removes CNAME based on the custom-domain input, stores credentials if provided, and returns the current target information. It configures the workspace, but authentication is handled by a stricter flow elsewhere.

writePublishAuthCache

writePublishAuthCache records a successful publishing authorization for the current target. It reads the previous cache first so it can preserve successful history when the repository, branch, and remote match.

Full source for writePublishAuthCache

```

async function writePublishAuthCache(access = {}) {

```

```

const previousCache = await readPublishAuthCache();
const checkedAt = new Date();
const checkedAtMs = checkedAt.getTime();
const sameTarget = previousCache
  && previousCache.remote === access.remote
  && previousCache.branch === access.branch
  && previousCache.repository === access.repository;
const previousHistory = sameTarget && Array.isArray(previousCache.successHistory)
  ? previousCache.successHistory
  : [];
const successTimestamps = [
  ...previousHistory
    .map((entry) => parsePublishAuthTimestamp(entry))
    .filter((timestamp) => Number.isFinite(timestamp))
    .filter((timestamp) => checkedAtMs - timestamp < publishAuthExtendedTtlMs),
  checkedAtMs
];
const successHistory = successTimestamps.map((timestamp) => new Date(timestamp).toISOString());
const successCountLastWeek = successTimestamps
  .filter((timestamp) => checkedAtMs - timestamp < publishAuthHistoryWindowMs)
  .length;
const extendedTrust = successCountLastWeek > publishAuthExtendedThreshold;
const ttlMs = extendedTrust ? publishAuthExtendedTtlMs : publishAuthCacheTtlMs;
const cache = {
  branch: access.branch || "",
  checkedAt: checkedAt.toISOString(),
  expiresAt: new Date(checkedAtMs + ttlMs).toISOString(),
  extendedTrust,
  remote: access.remote || "",
  repository: access.repository || "",
  scope: publishAuthCacheScope(),
  successCountLastWeek,
  successHistory,
  trustDays: extendedTrust ? 30 : 1
};
await writeFile(publishAuthCachePath, `${JSON.stringify(cache, null, 2)}\n`, "utf8");
await chmod(publishAuthCachePath, 0o600).catch(() => {});
return cache;
}

```

The function converts recent successes into timestamps, counts successes in the last week, and decides whether extended trust applies. It then writes branch, remote, repository, machine scope, checked time, expiration time, history, trust days, and extended-trust status to the cache file. Finally it tries to chmod the file to owner-only permissions.

This is the backend implementation of the convenience rule: normal success lasts one day, but repeated successful authorizations can be remembered longer on the same machine and workspace.

assertPublishAccess

assertPublishAccess is the gate in front of Apply to site. It first proves the publish workspace is a Git worktree, then reads origin and branch. If either is missing, it throws a publishAccessError with details the UI can display.

Full source for assertPublishAccess

```

async function assertPublishAccess(options = {}) {
  let insideWorkTree = "";
  try {
    insideWorkTree = (await runPublishGit(["rev-parse", "--is-inside-work-tree"])).stdout.trim();
  } catch (error) {
    throw publishAccessError(
      "This workspace is not connected to a Git repository.",
      gitFailureText(error),
      { repository: defaultSiteRepository }
    );
  }

  if (insideWorkTree !== "true") {
    throw publishAccessError(
      "This workspace is local-only and cannot publish yet.",
      "Clone or associate a GitHub Pages/static-site repository before applying to site.",
      { repository: defaultSiteRepository }
    );
  }

  let remote = "";

```

```

let branch = "";
try {
  remote = (await runPublishGit(["remote", "get-url", "origin"])).stdout.trim();
  branch = (await runPublishGit(["branch", "--show-current"])).stdout.trim();
} catch (error) {
  throw publishAccessError(
    "A publish remote or branch is missing.",
    gitFailureText(error),
    { repository: defaultSiteRepository }
  );
}

if (!remote || !branch) {
  throw publishAccessError(
    "A publish remote or branch is missing.",
    "Set the origin remote and use a named branch before applying to site.",
    { remote: remoteUrlForDisplay(remote), branch }
  );
}

if (options.requireCompatible !== false && !await workspaceHasCompatibleSiteFiles()) {
  throw publishAccessError(
    "This repository does not look like a compatible static portfolio website.",
    "The workspace must include index.html, styles.css, and script.js before it can be used as a publish target.",
    { remote: remoteUrlForDisplay(remote), branch }
  );
}

const parsedRemote = parseGitHubRemote(remote);
const repository = parsedRemote ? `${parsedRemote.owner}/${parsedRemote.repo}` : remoteUrlForDisplay(remote);

const access = {
  branch,
  remote: remoteUrlForDisplay(remote),
  repository,
  authorizationChecked: false
};

const cached = await readPublishAuthCache();
if (!options.force && publishAuthCacheIsFresh(cached, access)) {
  return {
    ...access,
    authorizationCached: true,
    authorizationChecked: true,
    checkedAt: cached.checkedAt,
    expiresAt: publishAuthCacheExpiresAt(cached),
    extendedTrust: Boolean(cached.extendedTrust),
    successCountLastWeek: cached.successCountLastWeek || 0,
    trustDays: cached.trustDays || (cached.extendedTrust ? 30 : 1)
  };
}

const remoteSync = await synchronizePublishBranchFromRemote(branch, access);
const localHead = await ensurePublishHeadForWriteCheck();
const writeCheck = await verifyPublishWriteAccess(access);

const authCache = await writePublishAuthCache(access);
return {
  ...access,
  remoteSync,
  localHead,
  writeCheck,
  authorizationChecked: true,
  authorizationCached: false,
  checkedAt: authCache.checkedAt,
  expiresAt: authCache.expiresAt,
  extendedTrust: Boolean(authCache.extendedTrust),
  successCountLastWeek: authCache.successCountLastWeek || 0,
  trustDays: authCache.trustDays || 1
};
}

```

After the basic Git shape is known, the function optionally checks that the workspace looks like a compatible static portfolio. Then it builds an access object containing branch, remote display URL, repository identity, and

authorization status. A fresh cache can satisfy the check immediately if it matches the same repository, branch, machine scope, and expiration window.

When the cache is absent or stale, the function synchronizes the branch from remote, makes sure a local HEAD exists for write checks, verifies write access, writes a new authorization cache, and returns a richer access object. This is the reason Apply to site can be strict without forcing a GitHub login on every launch.

syncFromPublishTarget

syncFromPublishTarget is the Load from target workflow. It starts by calling assertPublishAccess with compatibility disabled, because the point is to inspect the target repository and decide whether it contains compatible files.

Full source for syncFromPublishTarget

```
async function syncFromPublishTarget() {
  const publishAccess = await assertPublishAccess({ requireCompatible: false });
  const branch = publishAccess.branch || "main";
  const remote = (await runPublishGit(["remote", "get-url", "origin"])).stdout.trim();
  const importRoot = await mkdtemp(path.join(os.tmpdir(), "omb-publish-target-"));
  const cloneRoot = path.join(importRoot, "target");
  const importPaths = [
    "projects.json",
    "assets",
    "docs",
    "Backgrounds",
    "CNAME",
    ".nojekyll",
    "robots.txt",
    "sitemap.xml"
  ];

  try {
    await runGit(["clone", "--depth", "1", "--branch", branch, remote, cloneRoot], {
      cwd: importRoot,
      timeout: 3 * 60 * 1000
    });

    const availablePaths = [];
    for (const importPath of importPaths) {
      try {
        await fsAccess(path.join(cloneRoot, importPath));
        availablePaths.push(importPath);
      } catch {
        // Optional path is absent in the selected site repository.
      }
    }

    if (!availablePaths.includes("projects.json")) {
      throw publishAccessError(
        "The selected repository does not contain a projects.json portfolio catalog.",
        "The target repository must contain a compatible OMB Portfolio Builder catalog before it can be imported.",
        publishAccess
      );
    }

    const backupRoot = resolveInsideRoot(
      ".omb-backups",
      `load-target-${new Date().toISOString().replace(/[:.]/g, "-")}`
    );
    await mkdir(backupRoot, { recursive: true });

    for (const importPath of availablePaths) {
      const sourcePath = path.join(cloneRoot, importPath);
      const targetPath = resolveInsideRoot(importPath);
      try {
        await fsAccess(targetPath);
        await cp(targetPath, path.join(backupRoot, importPath), { recursive: true, force: true });
        await rm(targetPath, { recursive: true, force: true });
      } catch {
        // Nothing Local to back up for this path.
      }
      await cp(sourcePath, targetPath, { recursive: true, force: true });
    }
  }
}
```

```

const importedCatalog = await readFile(path.join(cloneRoot, "projects.json"), "utf8");
await writeFile(draftPath, importedCatalog, "utf8");
const portfolioSync = await syncPortfolioPublishFiles({ removeMissing: false });

return {
  ...publishAccess,
  imported: availablePaths,
  portfolioSync,
  backup: path.relative(root, backupRoot),
  draftUpdated: true
};
} finally {
  await rm(importRoot, { recursive: true, force: true });
}
}

```

The function clones the target branch into a temporary folder, collects available import paths, and requires projects.json. Before replacing local files, it creates a timestamped backup under .omb-backups. Then it copies compatible assets, docs, Backgrounds, CNAME, robots, sitemap, and the catalog into the local builder workspace.

The imported projects.json is also written to the local draft path, and syncPortfolioPublishFiles refreshes the publish mirror. The finally block removes the temporary clone, so the machine does not accumulate stale target copies after imports.

authenticateGitHubForTarget

authenticateGitHubForTarget is the interactive trust-building path. It validates the repository URL and optional domain, validates credentials, requires a remote URL, ensures the publish folder is a Git repository, and captures the current target state so it can roll back on failure.

Full source for authenticateGitHubForTarget

```

async function authenticateGitHubForTarget(options = {}) {
  const {
    repositoryUrl = "",
    customDomain = "",
    authUsername = "",
    authPassword = ""
  } = options;
  const customDomainProvided = Object.prototype.hasOwnProperty.call(options, "customDomain");
  const remoteUrl = validatePublishRemoteUrl(repositoryUrl);
  const domain = validateCustomDomain(customDomain);
  const credentials = validateCredentialPair(authUsername, authPassword);
  if (!remoteUrl) {
    throw publishAccessError(
      "A GitHub repository URL is required before authentication.",
      "Enter the GitHub Pages or compatible static-site repository URL, then click Authenticate target.",
      await getPublishTargetInfo()
    );
  }

  await ensureGitRepository();
  const snapshot = await capturePublishTargetState();

  try {
    await setOriginRemote(remoteUrl);
    const status = await getGitStatus();
    const preliminaryTarget = await getPublishTargetInfo();
    if (!status.git.ok) {
      throw publishAccessError(
        "Git for Windows is required for publishing.",
        "Install Git for Windows, then reopen the builder and authenticate again.",
        { ...preliminaryTarget, system: status }
      );
    }
  }

  const targetBranch = await detectRemoteDefaultBranch(remoteUrl) || "main";
  await checkoutPublishBranch(targetBranch);

  const targetBeforeAuth = await getPublishTargetInfo();
  const cached = await readPublishAuthCache();

```

```

const cachedAccess = {
  branch: targetBeforeAuth.branch,
  remote: targetBeforeAuth.remote,
  repository: targetBeforeAuth.repository
};
if (!credentials.cleanUsername && publishAuthCacheIsFresh(cached, cachedAccess)) {
  await writeTargetCustomDomain(domain, customDomainProvided);
  const target = await getPublishTargetInfo();
  return {
    ...cachedAccess,
    branch: target.branch || cachedAccess.branch,
    authorizationCached: true,
    authorizationChecked: true,
    checkedAt: cached.checkedAt,
    expiresAt: publishAuthCacheExpiresAt(cached),
    extendedTrust: Boolean(cached.extendedTrust),
    successCountLastWeek: cached.successCountLastWeek || 0,
    trustDays: cached.trustDays || (cached.extendedTrust ? 30 : 1),
    credentialsStored: false,
    credentialUsername: "",
    system: status,
    target
  };
}

if (!status.credentialManager.ok && !credentials.cleanUsername) {
  throw publishAccessError(
    "Git Credential Manager is required for guided GitHub sign-in.",
    "Install or repair Git for Windows with Git Credential Manager enabled, or provide a GitHub username and personal access token.",
    { ...targetBeforeAuth, system: status }
  );
}

const storedCredentials = await storeGitCredentials(remoteUrl, authUsername, authPassword);
if (!storedCredentials.stored) {
  await runGit(["credential-manager", "github", "login"], {
    cwd: portfolioRoot,
    timeout: 5 * 60 * 1000,
    windowsHide: false
  });
}

const access = await assertPublishAccess({ force: true, requireCompatible: false });
await writeTargetCustomDomain(domain, customDomainProvided);
const target = await getPublishTargetInfo();
return {
  ...access,
  branch: target.branch || access.branch,
  credentialsStored: storedCredentials.stored,
  credentialUsername: storedCredentials.username || "",
  system: await getGitStatus(),
  target
};
} catch (error) {
  await restorePublishTargetState(snapshot);
  throw error;
}
}

```

Inside the try block, it sets the origin remote, checks Git/Git Credential Manager health, detects the target branch, and checks out that branch. If a matching authorization cache is already fresh and no new credentials were supplied, it can return cached success after writing the custom domain.

If credentials are needed, the function either stores the supplied username/token pair or launches Git Credential Manager's GitHub login. It then forces `assertPublishAccess`, writes the custom domain, returns target/system details, and restores the previous target state if any error interrupts the sequence.

syncPortfolioPublishFiles

`syncPortfolioPublishFiles` copies publishable website files from the builder workspace to the portfolio mirror. When root and portfolioRoot are the same folder, there is nothing to copy and the function reports that the workspace is not separated.

Full source for syncPortfolioPublishFiles

```
async function syncPortfolioPublishFiles(options = {}) {
  if (samePath(root, portfolioRoot)) {
    return { copied: [], skipped: [], workspace: portfolioRoot, separatedWorkspace: false };
  }

  const { removeMissing = false } = options;
  const copied = [];
  const skipped = [];

  await mkdir(portfolioRoot, { recursive: true });

  for (const relativePath of publishPaths) {
    const sourcePath = resolveInsideRoot(relativePath);
    const targetPath = resolveInsidePortfolioRoot(relativePath);
    if (await pathExists(sourcePath)) {
      await rm(targetPath, { recursive: true, force: true });
      await mkdir(path.dirname(targetPath), { recursive: true });
      await cp(sourcePath, targetPath, { recursive: true, force: true });
      copied.push(relativePath);
    } else if (removeMissing) {
      await rm(targetPath, { recursive: true, force: true });
      skipped.push(relativePath);
    } else {
      skipped.push(relativePath);
    }
  }

  return {
    copied,
    skipped,
    workspace: portfolioRoot,
    separatedWorkspace: true
  };
};
```

In a separated setup, the function loops only through publishPaths. Existing source paths are copied after the destination is cleared. Missing paths can be removed from the mirror when removeMissing is true. That behavior keeps the public mirror aligned with the current builder output without staging arbitrary folders.

publishSiteChanges

publishSiteChanges is the final Apply to site operation. It starts with a publishAccess object if one was already created, otherwise it calls assertPublishAccess itself. Then it synchronizes the remote branch, copies publishable files into the portfolio mirror, and bumps published asset versions.

Full source for publishSiteChanges

```
async function publishSiteChanges(publishAccess = null) {
  const access = publishAccess || await assertPublishAccess();
  const branch = access.branch || (await runPublishGit(["branch", "--show-current"])).stdout.trim();
  const remoteSync = await synchronizePublishBranchFromRemote(branch, access);
  const portfolioSync = await syncPortfolioPublishFiles({ removeMissing: true });
  await bumpPublishedAssetVersions(portfolioRoot);

  const stageablePaths = await stageablePublishPaths(publishPaths);
  if (!stageablePaths.length) {
    throw new Error("No compatible site files were available to publish.");
  }

  await runPublishGit(["add", "-A", "--", ...stageablePaths]);
  const status = await runPublishGit(["status", "--porcelain", "--", ...stageablePaths]);
  const hasChanges = status.stdout.trim().length > 0;

  let commit = null;
  if (hasChanges) {
    const message = `Update portfolio site ${new Date().toISOString().slice(0, 10)}`;
    commit = await runPublishGit(["commit", "-m", message]);
  }

  const pushArgs = branch ? ["push", "origin", branch] : ["push"];
  const push = await runPublishGit(pushArgs);
}
```

```

return {
  ...access,
  portfolioWorkspace: portfolioRoot,
  remoteSync,
  portfolioSync,
  branch: branch || "current branch",
  committed: hasChanges,
  commitOutput: commit?.stdout || commit?.stderr || "",
  pushed: true,
  pushOutput: push.stdout || push.stderr || ""
};
}

```

The function asks `stageablePublishPaths` which allowlisted paths actually exist, stages those paths with `git add`, checks status only for those paths, and commits only when there are changes. The commit message is date-based because the action is a generated portfolio update, not a hand-authored feature commit.

Finally it pushes to the current branch and returns a public-facing publish report: workspace path, sync details, branch, whether a commit happened, commit output, push status, and push output.

Chapter 6: AI Context, Public Source Fetching, And Model Routing

Ask My Portfolio is stronger when it can separate a greeting from a portfolio question, fetch safe public source evidence, and choose an AI backend without exposing private keys to the browser.

Important working state

These objects and constants are shown before the functions because they are the shared state the functions read from or protect. They are not syntax trivia; they explain what the backend remembers while it runs.

portfolioAiInstructions

portfolioAiInstructions is the policy prompt for Ask My Portfolio. It tells the model how to separate general questions from portfolio-specific questions, how to use public source excerpts, when to cite GitHub code, and what not to invent.

Server state excerpt: portfolioAiInstructions

```
const portfolioAiInstructions = [
  "You are the AI assistant for the portfolio website described by the supplied portfolio context.",
  "Behave like a careful senior electrical and computer engineering mentor who can also navigate the saved portfolio.",
  "Answer the visitor's question first in a concise ChatGPT-like format, then add portfolio links or context only when they help.",
  "Use the supplied question intent to decide whether this is a general engineering question or a portfolio-specific question.",
  "For general_conversation intent, respond naturally and briefly. Use the recent conversation instead of a fixed template. If the visitor says hi, a good answer is a short greeting such as: Hi, what can I do for you?",
  "If the visitor asks 'what is my name?' or 'who am I?' and they have not identified themselves, do not pretend to know the visitor. Say you are an AI agent for this portfolio and identify the portfolio owner only if the supplied profile context includes a name.",
  "For general_knowledge intent, answer the question directly using broad general knowledge. Do not force portfolio context unless the visitor asks to connect the answer to the portfolio owner's work.",
  "For general_engineering intent, begin with the general electronics or engineering explanation. Do not lead with project context unless the visitor asks to connect it.",
  "For portfolio_specific intent, answer from the portfolio context first, then explain related engineering concepts only when useful.",
  "Use recent conversation history for follow-up questions, pronouns, comparisons, and corrections.",
  "Use the supplied portfolio context as the trusted source for the portfolio owner's projects, links, files, resume, and contact information.",
  "Use sourceExcerpts when present as higher-detail evidence from uploaded text files, extracted resume text, same-site files, GitHub pages, or other safe public sources.",
  "Use knowledgeManifest to understand project files, image evidence, public profiles, resumes, and project areas. Treat filenames, captions, surrounding text, and descriptions as evidence.",
  "Do not claim to visually inspect an image unless actual image analysis is provided. If only image metadata is supplied, say what the caption/path/context suggests.",
  "For GitHub, LinkedIn, resume, or uploaded-file questions, cite what is present in the supplied context or fetched excerpts and then point to the link when useful. Access public pages and fetched excerpts when the website allows it.",
  "When a visitor asks for source code, use public GitHub source excerpts when provided. Show concise relevant snippets with file paths, and explain what the code is doing. Do not imply private repository access.",
  "If sourceExcerpts include lines labeled Source file, include at least one fenced code block with the file path immediately before it, unless the visitor explicitly asks for links only.",
  "Never invent, infer, or write hypothetical code for the portfolio owner's repositories. Every fenced code block about repository work must be copied from a fetched Source file excerpt. If no source file excerpt is available, say that the code was not available in the fetched public sources and give the repository link instead.",
  "For questions about public GitHub repositories, prefer fetched public GitHub source excerpts over portfolio summaries or project descriptions. You may fetch and display concise code from public GitHub URLs supplied in the portfolio context or safe source fetches, but never imply access to private repositories.",
  "You may answer related electronics, hardware, analog, mixed-signal, digital, embedded, FPGA, ASIC, PCB, and firmware questions even when the answer is broader than the saved portfolio.",
  "Do not invent portfolio projects, credentials, employers, files, or test results that are not in the context.",
  "If context is missing, say what is missing and answer generally only for the engineering concept.",
  "When useful, state assumptions, define terms, explain signal or data flow, identify tradeoffs, and name what evidence would prove the claim.",
  "Only mention or recommend portfolio links that are directly relevant to the visitor's current question or the active follow-up context. Do not append random links.",
  "Keep the answer recruiter-friendly, specific, and easy to skim. Use short paragraphs and bullets when helpful.",
  "Do not expose chain-of-thought. Give the polished answer only."
].join("\n");
```

Function walkthrough

fetchGitHubRepositorySource

fetchGitHubRepositorySource is the public-code collector for Ask My Portfolio. It starts by reading repository metadata through GitHub's API, chooses a branch, and derives limits for how many files and how much text can enter the AI context.

Full source for fetchGitHubRepositorySource

```
async function fetchGitHubRepositorySource(source = {}, parsed = {}) {
  const { owner, repo } = parsed;
  const metadata = await fetchGitHubJson(`https://api.github.com/repos/${encodeURIComponent(owner)}/${encodeURIComponent(repo)}`);
  const branch = parsed.branch || metadata?.default_branch || "main";
  const question = source.question || "";
  const requestedMaxCodeFiles = Number(source.maxCodeFiles || (wantsGitHubCode(question) ? 6 : 3));
  const maxCodeFiles = Math.max(1, Math.min(8, Number.isFinite(requestedMaxCodeFiles) ? requestedMaxCodeFiles : 3));
  const requestedFileTextLength = Number(source.maxFileTextLength || (wantsGitHubCode(question) ? 7000 : 3000));
  const maxFileTextLength = Math.max(1000, Math.min(10000, Number.isFinite(requestedFileTextLength) ? requestedFileTextLength : 3000));
  const requestedRepoTextLength = Number(source.maxRepositoryTextLength || (wantsGitHubCode(question) ? 22000 : 14000));
  const maxRepositoryTextLength = Math.max(5000, Math.min(26000, Number.isFinite(requestedRepoTextLength) ? requestedRepoTextLength : 14000));
  const lines = [
    `GitHub repository: ${owner}/${repo}`,
    metadata?.description ? `Description: ${metadata.description}` : "",
    metadata?.language ? `Primary language: ${metadata.language}` : "",
    metadata?.html_url ? `Repository URL: ${metadata.html_url}` : source.url,
    metadata?.default_branch ? `Default branch: ${metadata.default_branch}` : "",
    ""
  ].filter(Boolean);

  if (parsed.type === "file" && parsed.filePath) {
    const rawUrl = `https://raw.githubusercontent.com/${owner}/${repo}/${branch}/${parsed.filePath}`;
    const text = await fetchLimitedText(rawUrl, 14000);
    if (text.trim()) lines.push(`Source file: ${parsed.filePath}`, text.trim());
    return {
      context: source.context || "Public GitHub source file",
      title: source.title || source.label || `${owner}/${repo}/${parsed.filePath}`,
      type: "public GitHub source file",
      url: source.url,
      text: clampText(lines.join("\n"), 14000)
    };
  }

  const tree = await fetchGitHubJson(`https://api.github.com/repos/${encodeURIComponent(owner)}/${encodeURIComponent(repo)}/git/trees/${encodeURIComponent(branch)}?recursive=1`);
  const files = Array.isArray(tree?.tree)
    ? tree.tree
      .filter((item) => item.type === "blob" && item.path && githubTextFilePattern.test(item.path) && !githubSkipFilePattern.test(item.path))
      .sort((a, b) => scoreGitHubFile(b.path, question) - scoreGitHubFile(a.path, question))
    : [];

  const readmePath = files.find((file) => /^(^|\/)readme\.md$/i.test(file.path)?.path || "README.md");
  const readmeText = await fetchLimitedText(`https://raw.githubusercontent.com/${owner}/${repo}/${branch}/${readmePath}`, 9000).catch(() => "");
  if (readmeText.trim()) lines.push(`README excerpt from ${readmePath}:`, readmeText.trim(), "");

  if (files.length) {
    lines.push(`Repository text/code file list:`, ...files.slice(0, 30).map((file) => ` - ${file.path}`), "");
  }

  const includeCode = wantsGitHubCode(question);
  const selectedFiles = files
    .filter((file) => !/readme\.md$/i.test(file.path))
    .slice(0, maxCodeFiles);

  for (const file of selectedFiles) {
    const rawUrl = `https://raw.githubusercontent.com/${owner}/${repo}/${branch}/${file.path}`;
    const fileText = await fetchLimitedText(rawUrl, maxFileTextLength).catch(() => "");
    if (!fileText.trim()) continue;
  }
}
```

```

    lines.push(`Source file: ${file.path}`, fileText.trim(), "");
  }

  return {
    context: source.context || "Public GitHub repository",
    title: source.title || source.label || `${owner}/${repo}`,
    type: includeCode ? "public GitHub repository and code" : "public GitHub repository",
    url: source.url,
    text: clampText(lines.join("\n"), maxRepositoryTextLength)
  };
}

```

If the URL points directly to a file, the function fetches that raw file and returns a source excerpt immediately. Otherwise it reads the repository tree, filters to text-like files, scores files against the visitor's question, pulls README text, lists candidate files, and fetches selected source files. The final text is clamped so a repository cannot flood the prompt.

This function is why the assistant can answer code questions from public GitHub repositories while still respecting boundaries. It fetches public evidence; it does not claim private access.

enrichPortfolioContext

enrichPortfolioContext receives the context packet prepared by the public site and adds safe fetched excerpts. It copies up to ten sourceFetches, attaches the current question to each one, and runs fetchSourceText for each source.

Full source for enrichPortfolioContext

```

async function enrichPortfolioContext(context = {}) {
  const question = String(context.question || "");
  const sources = Array.isArray(context.sourceFetches)
    ? context.sourceFetches.slice(0, 10).map((source) => ({ ...source, question: source.question || question }))
    : [];
  const sourceExcerpts = (await Promise.all(sources.map(fetchSourceText))).filter(Boolean);
  return {
    ...context,
    sourceExcerpts,
    sourceFetchPolicy: "Fetched excerpts are limited to safe text-like same-site, GitHub/raw GitHub, LinkedIn, and public portfolio URLs. GitHub repository links can be expanded into repository metadata, README text, and selected public source files when a question asks for code. PDFs/images are represented by captions, metadata, filenames, and companion text files when available."
  };
}

```

The returned context keeps everything from the original packet and adds sourceExcerpts plus a sourceFetchPolicy string. That policy is important because the model receives not only evidence but also the rule explaining what kind of evidence was fetched and what kind was only represented by metadata.

callOllamaPortfolioAi

callOllamaPortfolioAi is the local-model fallback. It chooses the Ollama host and model from environment variables, creates an AbortController, and sets a timeout so a missing or slow local model does not freeze the API request.

Full source for callOllamaPortfolioAi

```

async function callOllamaPortfolioAi({ question, intent, conversation, context, allowWebSearch }) {
  const host = process.env.OLLAMA_HOST || "http://127.0.0.1:11434";
  const model = process.env.OLLAMA_MODEL || "llama3.2";
  const controller = new AbortController();
  const timeout = setTimeout(() => controller.abort(), Number(process.env.OLLAMA_TIMEOUT_MS || 45000));
  try {
    const response = await fetch(`${host.replace(/\/\+$/, "")}/api/chat`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      signal: controller.signal,
      body: JSON.stringify({
        model,
        stream: false,
        messages: [
          { role: "system", content: portfolioAiInstructions },
          {
            role: "user",
            content: [

```

```

        `Visitor question: ${question}`,
        `Question intent: ${intent}`,
        `Web/public lookup requested by browser: ${allowWebSearch ? "yes" : "no"}`,
        "",
        "Recent conversation JSON:",
        clampText(JSON.stringify(conversation, null, 2), 8000),
        "",
        "Portfolio context JSON:",
        clampText(JSON.stringify(context, null, 2), 18000)
    ].join("\n")
    }
    ],
    options: {
        temperature: Number(process.env.OLLAMA_TEMPERATURE || 0.35)
    }
    })
    });
    const data = await response.json().catch(() => ({}));
    if (!response.ok) {
        return { ok: false, error: data?.error || `Ollama returned HTTP ${response.status}.` };
    }
    const answer = extractOllamaText(data);
    return answer
        ? { ok: true, answer, model }
        : { ok: false, error: "Ollama did not return an answer." };
    } catch (error) {
        return { ok: false, error: error?.name === "AbortError" ? "Ollama timed out." : "Ollama is not reachable on this machine." };
    } finally {
        clearTimeout(timeout);
    }
    }
}

```

The function sends a chat request containing the portfolio AI instructions, question, intent, conversation, context, and web-search flag. It reads the response JSON, extracts text through `extractOllamaText`, and returns a normalized ok/error object. If the request fails or times out, the caller receives a clean failure instead of an unhandled network exception.

handlePortfolioAi

`handlePortfolioAi` is the backend API endpoint for Ask My Portfolio. It reads the request JSON, clamps the question, normalizes conversation history, validates the intent, and rejects empty questions before doing any model work.

Full source for handlePortfolioAi

```

async function handlePortfolioAi(request, response) {
    const body = await readRequestJson(request);
    const question = clampText(body.question, 1200).trim();
    const conversation = cleanConversationHistory(body.conversation);
    const validIntents = new Set(["general_conversation", "general_engineering", "general_knowledge", "portfolio_specific"]);
    const intent = validIntents.has(body.intent) ? body.intent : "portfolio_specific";
    const allowWebSearch = body.allowWebSearch === true;

    if (!question) {
        sendJson(response, 400, { error: "Question is required." });
        return;
    }

    const context = await enrichPortfolioContext({ ...(body.context || {}), question });

    const apiKey = process.env.OPENAI_API_KEY || "";
    if (!apiKey) {
        const ollama = await callOllamaPortfolioAi({ question, intent, conversation, context, allowWebSearch });
        if (ollama.ok) {
            sendJson(response, 200, {
                answer: ollama.answer,
                model: ollama.model,
                provider: "ollama",
                usedWebSearch: false
            });
            return;
        }
    }
}

```

```

    sendJson(response, 503, { error: ollama.error || "No local Ollama model or OPENAI_API_KEY is available for the
local backend." });
    return;
}

const model = process.env.OPENAI_MODEL || "gpt-5.4";
const fallbackModel = process.env.OPENAI_FALLBACK_MODEL || "gpt-4.1";
const webSearchMode = String(process.env.OPENAI_ENABLE_WEB_SEARCH || "auto").toLowerCase();
const enableWebSearch = webSearchMode === "true" || (webSearchMode !== "false" && allowWebSearch);
const buildPayload = (selectedModel) => ({
  model: selectedModel,
  input: [
    {
      role: "developer",
      content: [{ type: "input_text", text: portfolioAiInstructions }]
    },
    {
      role: "user",
      content: [{
        type: "input_text",
        text: [
          `Visitor question: ${question}`,
          `Question intent: ${intent}`,
          `Web search allowed for this question: ${enableWebSearch ? "yes" : "no"}`,
          "",
          "Recent conversation JSON:",
          clampText(JSON.stringify(conversation, null, 2), 8000),
          "",
          "Portfolio context JSON:",
          clampText(JSON.stringify(context, null, 2), 18000)
        ].join("\n")
      ]
    }
  ]
});

max_output_tokens: Number(process.env.OPENAI_MAX_OUTPUT_TOKENS || 1100),
reasoning: { effort: process.env.OPENAI_REASONING_EFFORT || "medium" },
text: { verbosity: process.env.OPENAI_VERBOSITY || "medium" }
});

const callModel = async (selectedModel) => {
  const payload = buildPayload(selectedModel);
  if (enableWebSearch) {
    payload.tools = [{ type: "web_search", search_context_size: process.env.OPENAI_WEB_SEARCH_CONTEXT_SIZE ||
"low" }];
  }

  const openAiResponse = await fetch("https://api.openai.com/v1/responses", {
    method: "POST",
    headers: {
      "Authorization": `Bearer ${apiKey}`,
      "Content-Type": "application/json"
    },
    body: JSON.stringify(payload)
  });

  const data = await openAiResponse.json().catch(() => ({}));
  return { data, openAiResponse, selectedModel };
};

let { data, openAiResponse, selectedModel } = await callModel(model);
if (!openAiResponse.ok && !process.env.OPENAI_MODEL && fallbackModel && fallbackModel !== model && [400,
404].includes(openAiResponse.status)) {
  ({ data, openAiResponse, selectedModel } = await callModel(fallbackModel));
}
if (!openAiResponse.ok) {
  sendJson(response, openAiResponse.status, {
    error: data?.error?.message || "OpenAI request failed."
  });
  return;
}

sendJson(response, 200, {
  answer: extractOpenAiText(data),
  model: selectedModel,
  usedWebSearch: enableWebSearch
});

```

```
}
```

The next step enriches portfolio context. After context is prepared, the function chooses a provider. If `OPENAI_API_KEY` is missing, it tries Ollama and returns a 503 only when the local model path also fails. If OpenAI is configured, it builds a Responses API payload with developer instructions, visitor question, intent, conversation JSON, portfolio context JSON, model settings, and optional web search.

The function also handles fallback model selection for default OpenAI model errors. A successful response returns answer text, model, and web-search status. A failed response returns the provider error through `sendJson` so the frontend can show a clear AI-backend problem.

Chapter 7: Updates, Security Reporting, And The API Switchboard

The final server responsibilities are maintenance and routing. The app needs update checks, security visibility, and one central API switchboard that maps browser actions to backend functions.

Function walkthrough

getUpdateInfo

getUpdateInfo checks whether a newer builder release exists. It reads the current version from the app environment or package.json, calls the GitHub latest-release API, extracts the release tag, finds installer and portable assets, and compares versions.

Full source for getUpdateInfo

```
async function getUpdateInfo() {
  const currentVersion = process.env.OMB_APP_VERSION || packageJson.version || "0.0.0";
  const owner = "otienomaurice";
  const repo = "omb-portfolio-builder";
  try {
    const response = await fetch(`https://api.github.com/repos/${owner}/${repo}/releases/latest`, {
      headers: { "Accept": "application/vnd.github+json", "User-Agent": "OMB-Portfolio-Builder" }
    });
    if (!response.ok) throw new Error(`GitHub returned ${response.status}.`);
    const release = await response.json();
    const latestVersion = String(release.tag_name || "").replace(/^builder-v/i, "");
    const installer = (release.assets || []).find((asset) => /Setup-.*\.exe$/i.test(asset.name));
    const portable = (release.assets || []).find((asset) => /Portable-.*\.exe$/i.test(asset.name));
    const updateBlocked = blockedAppUpdateVersions.has(latestVersion);
    return {
      ok: true,
      currentVersion,
      latestVersion,
      updateAvailable: !updateBlocked && compareVersions(latestVersion, currentVersion) > 0,
      updateBlocked,
      blockedReason: updateBlocked
        ? `Builder ${latestVersion} is skipped because its installer can hang while running the old uninstaller during in-app updates. Wait for builder 0.2.17 or newer, then run Update again.`
        : "",
      releaseUrl: release.html_url || "",
      installerUrl: installer?.browser_download_url || "",
      installerName: installer?.name || "",
      portableUrl: portable?.browser_download_url || ""
    };
  } catch (error) {
    return {
      ok: false,
      currentVersion,
      latestVersion: currentVersion,
      updateAvailable: false,
      error: error.message || "Could not check for updates."
    };
  }
}
```

The function also checks blockedAppUpdateVersions. A release can exist but be deliberately skipped if a known updater issue would make automatic installation unsafe. The returned object gives the UI enough information to show up-to-date, update-available, blocked, or error states.

getSecurityReport

getSecurityReport produces the builder's security visibility report. It runs release-download counts, authorization cache reads, and target info reads in parallel with Promise.allSettled so one failed source does not destroy the whole report.

Full source for getSecurityReport

```
async function getSecurityReport() {
  const [downloadResult, authCacheResult, targetResult] = await Promise.allSettled([
    getBuilderReleaseDownloadReport(),
    readPublishAuthCache(),
    getPublishTargetInfo()
  ]);
}
```

```

});
const authCache = authCacheResult.status === "fulfilled" ? authCacheResult.value : null;
const target = targetResult.status === "fulfilled" ? targetResult.value : null;
return {
  ok: true,
  generatedAt: new Date().toISOString(),
  appDownloads: downloadResult.status === "fulfilled"
    ? { ok: true, ...downloadResult.value }
    : { ok: false, error: downloadResult.reason?.message || "GitHub release download counts were not available."
},
  publishingAuth: {
    targetRepository: target?.repository || "",
    branch: target?.branch || "",
    authenticated: Boolean(authCache && publishAuthCacheIsFresh(authCache, {
      remote: target?.remote || "",
      repository: target?.repository || "",
      branch: target?.branch || ""
    })),
    expiresAt: publishAuthCacheExpiresAt(authCache) || "",
    trustDays: Number(authCache?.trustDays || 0),
    successCountLastWeek: Number(authCache?.successCountLastWeek || 0)
  },
  websiteAccess: {
    available: false,
    summary: "Visitor counts and visitor IP addresses are not available from a static GitHub Pages site or GitHub
release download counts.",
    recommendedSetup: [
      "Proxy the custom domain through Cloudflare instead of DNS-only.",
      "Enable Cloudflare Web Analytics for aggregate page-view counts.",
      "Use Cloudflare Logpush, Analytics Engine, or a Worker-backed endpoint if you need IP-level security logs.",
      "Publish a privacy notice before collecting raw IP addresses or persistent identifiers."
    ]
  },
  securityControls: [
    "Local builder write APIs are restricted to local requests.",
    "Publishing target details are only returned to the local machine.",
    "GitHub publishing authorization is cached for one day by default, or longer after repeated successful
authorizations.",
    "The desktop app runs with Electron nodeIntegration disabled, contextIsolation enabled, and sandbox enabled.",
    "Deployable website security headers and security.txt metadata are included in the publishable site files."
  ]
};
}

```

The returned report is intentionally honest. GitHub release assets can reveal download counts, and the local cache can reveal publishing-auth state, but a static GitHub Pages website cannot identify every visitor or provide raw IP logs by itself. That is why the report recommends Cloudflare analytics, Logpush, Analytics Engine, or a Worker-backed endpoint for IP-level logging.

downloadAndLaunchAppUpdate

downloadAndLaunchAppUpdate performs the in-app updater handoff. It starts by calling getUpdateInfo and refuses to continue when the current version is already current, the release is blocked, no installer is available, or the OS is not Windows.

Full source for downloadAndLaunchAppUpdate

```

async function downloadAndLaunchAppUpdate() {
  const update = await getUpdateInfo();
  if (!update.ok) throw new Error(update.error || "Could not check for updates.");
  if (update.updateBlocked) {
    throw new Error(update.blockedReason || `Builder ${update.latestVersion} is temporarily blocked from automatic
updates.`);
  }
  if (!update.updateAvailable) {
    throw new Error(`OMB Portfolio Builder ${update.currentVersion || ""} is already up to date.`);
  }
  if (!update.installerUrl) {
    throw new Error("The latest GitHub Release does not include a Windows setup installer.");
  }
  if (process.platform !== "win32") {
    throw new Error("Automatic installer updates are currently available only on Windows.");
  }

  const updateRoot = path.join(os.tmpdir(), "omb-portfolio-builder-updates");
  await mkdir(updateRoot, { recursive: true });
}

```

```

const installerName = update.installerName && /\.exe$/i.test(update.installerName)
  ? update.installerName
  : `OMB-Portfolio-Builder-Setup-${safeUpdateFileSegment(update.latestVersion)}-x64.exe`;
const installerPath = path.join(updateRoot, installerName);

const response = await fetch(update.installerUrl, {
  headers: { "Accept": "application/octet-stream", "User-Agent": "OMB-Portfolio-Builder" }
});
if (!response.ok) throw new Error(`GitHub returned ${response.status} while downloading the installer.`);
const installerBuffer = Buffer.from(await response.arrayBuffer());
if (installerBuffer.length < 1024 * 1024) {
  throw new Error("The downloaded installer was unexpectedly small, so the update was stopped.");
}
await writeFile(installerPath, installerBuffer);

const updateLogPath = path.join(updateRoot, `update-${safeUpdateFileSegment(update.latestVersion)}.log`);
const currentExecutable = /OMB Portfolio Builder\.exe$/i.test(process.execPath || "") ? process.execPath : "";
const currentInstallDirectory = currentExecutable ? path.dirname(currentExecutable) : "";
const localAppDataExecutable = process.env.LOCALAPPDATA
  ? path.join(process.env.LOCALAPPDATA, "Programs", "OMB Portfolio Builder", "OMB Portfolio Builder.exe")
  : "";
const legacyManagedApplicationExecutable = path.join(os.homedir(), "OMB", "application", "OMB Portfolio Builder",
"OMB Portfolio Builder.exe");
const legacyFlatApplicationExecutable = path.join(os.homedir(), "OMB", "application", "OMB Portfolio
Builder.exe");
const relaunchCandidates = Array.from(new Set([
  localAppDataExecutable,
  currentExecutable,
  currentInstallDirectory ? path.join(currentInstallDirectory, "OMB Portfolio Builder.exe") : "",
  legacyFlatApplicationExecutable,
  legacyManagedApplicationExecutable,
  process.env.ProgramFiles ? path.join(process.env.ProgramFiles, "OMB Portfolio Builder", "OMB Portfolio
Builder.exe") : "",
  process.env["ProgramFiles(x86)"] ? path.join(process.env["ProgramFiles(x86)"], "OMB Portfolio Builder", "OMB
Portfolio Builder.exe") : ""
].filter(Boolean)));
const relaunchCandidatesPs = `@(${relaunchCandidates.map(powershellSingleQuoted).join(", ")});`;
const launcherPath = path.join(updateRoot, `run-update-${safeUpdateFileSegment(update.latestVersion)}.ps1`);
const launcherCommandPath = path.join(updateRoot, `run-update-${
safeUpdateFileSegment(update.latestVersion)}.cmd`);
const launcherScript = [
  "$ErrorActionPreference = 'Continue'",
  `$parentPid = ${process.pid}`,
  `$installer = ${powershellSingleQuoted(installerPath)}`,
  `$log = ${powershellSingleQuoted(updateLogPath)}`,
  `$candidates = ${relaunchCandidatesPs}`,
  "function Write-UpdateLog([string]$Message) {",
  "  try { Add-Content -LiteralPath $log -Value ((Get-Date).ToString('s') + ' ' + $Message) } catch {}",
  "}",
  "function Wait-ForExecutable([string[]]$Paths, [int]$Seconds) {",
  "  $deadline = (Get-Date).AddSeconds($Seconds)",
  "  do {",
  "    foreach ($candidate in $Paths) {",
  "      if ($candidate -and (Test-Path -LiteralPath $candidate)) { return $candidate }",
  "    }",
  "    Start-Sleep -Seconds 1",
  "  } while ((Get-Date) -lt $deadline)",
  "  return $null",
  "}",
  "function Stop-BuilderProcesses([int]$ProcessId, [string[]]$Paths) {",
  "  try {",
  "    $normalizedPaths = @($Paths | Where-Object { $_ } | ForEach-Object {",
  "      try { [System.IO.Path]::GetFullPath($_).ToLowerInvariant() } catch { $_.ToLowerInvariant() }",
  "    })",
  "    $processes = Get-CimInstance Win32_Process | Where-Object {",
  "      $_.ProcessId -eq $ProcessId -or",
  "      $_.Name -eq 'OMB Portfolio Builder.exe' -or",
  "      ($_.ExecutablePath -and ($normalizedPaths -contains",
  "([System.IO.Path]::GetFullPath($_.ExecutablePath).ToLowerInvariant()))",
  "    } | Sort-Object ProcessId -Unique",
  "    foreach ($process in $processes) {",
  "      try {",
  "        Write-UpdateLog ('Stopping previous builder process PID ' + $process.ProcessId + ' at ' +
$process.ExecutablePath)",
  "        Stop-Process -Id $process.ProcessId -Force -ErrorAction SilentlyContinue",
  "      } catch { Write-UpdateLog ('Could not stop PID ' + $process.ProcessId + ': ' + $_.Exception.Message) }",
  "    }",
  "  }",

```

```

        "    Start-Sleep -Seconds 2",
        " } catch { Write-UpdateLog ('Could not stop previous builder processes: ' + $_.Exception.Message) }",
        "}",
        "function Run-Installer([switch]$Elevated) {",
        "    $process = $null",
        "    if ($Elevated) {",
        "        Write-UpdateLog 'Starting installer with Windows elevation prompt.'",
        "        $process = Start-Process -FilePath $installer -ArgumentList @('/S') -Verb RunAs -PassThru",
        "    } else {",
        "        Write-UpdateLog 'Starting installer silently.'",
        "        $process = Start-Process -FilePath $installer -ArgumentList @('/S') -WindowStyle Hidden -PassThru",
        "    }",
        "    if (-not $process) { return 0 }",
        "    if (-not $process.WaitForExit(900000)) {",
        "        Write-UpdateLog ('Installer timed out after 15 minutes. PID: ' + $process.Id)",
        "        try { Stop-Process -Id $process.Id -Force -ErrorAction SilentlyContinue } catch { Write-UpdateLog ('Could not stop timed-out installer: ' + $_.Exception.Message) }",
        "        return 124",
        "    }",
        "    return $process.ExitCode",
        "}",
        "Write-UpdateLog 'Update launcher started.'",
        "Start-Sleep -Seconds 3",
        "try { Wait-Process -Id $parentPid -Timeout 8; Write-UpdateLog 'Previous app process exited.' } catch { Write-UpdateLog ('Previous app wait finished: ' + $_.Exception.Message) }",
        "Stop-BuilderProcesses $parentPid $candidates",
        "Start-Sleep -Seconds 1",
        "$installerProcess = $null",
        "$installerExit = 1",
        "try {",
        "    Write-UpdateLog ('Starting installer: ' + $installer)",
        "    $installerExit = Run-Installer",
        "    Write-UpdateLog ('Installer exit code: ' + $installerExit)",
        "} catch {",
        "    Write-UpdateLog ('Installer launch failed: ' + $_.Exception.Message)",
        "    exit 1",
        "}",
        "if ($installerExit -ne 0) {",
        "    try {",
        "        Write-UpdateLog 'Installer did not finish cleanly; retrying with Windows elevation prompt.'",
        "        $installerExit = Run-Installer -Elevated",
        "        Write-UpdateLog ('Elevated installer exit code: ' + $installerExit)",
        "    } catch { Write-UpdateLog ('Elevated installer launch failed: ' + $_.Exception.Message) }",
        "}",
        "if ($installerExit -ne 0) { exit $installerExit }",
        "$target = Wait-ForExecutable $candidates 90",
        "if ($target) {",
        "    try {",
        "        Start-Sleep -Seconds 2",
        "        Start-Process -FilePath $target -WorkingDirectory (Split-Path -Parent $target)",
        "        Write-UpdateLog ('Relaunched app: ' + $target)",
        "        exit 0",
        "    } catch {",
        "        Write-UpdateLog ('Relaunch failed: ' + $_.Exception.Message)",
        "        exit 3",
        "    }",
        "}",
        "Write-UpdateLog 'No installed app executable was found after update.'",
        "exit 2"
    ].join("`r`n");
    await writeFile(launcherPath, launcherScript, "utf8");
    const launcherCommandScript = [
        "@echo off",
        `start "" /min powershell.exe -NoProfile -ExecutionPolicy Bypass -WindowStyle Hidden -File "${launcherPath}"`
    ].join("`r`n");
    await writeFile(launcherCommandPath, launcherCommandScript, "utf8");

    const child = spawn("cmd.exe", ["/d", "/c", launcherCommandPath], {
        detached: true,
        stdio: "ignore",
        windowsHide: true
    });
    child.unref();

    setTimeout(() => {
        process.emit("omb-builder-update-started", { installerPath, launcherPath, updateLogPath });
    }, 600);

```

```

setTimeout(() => {
  process.exit(0);
}, 3000);

return {
  ...update,
  installerPath,
  launcherPath,
  updateLogPath,
  launcherCommandPath,
  started: true
};
}

```

After those checks, it downloads the installer into a temp update folder, rejects suspiciously tiny downloads, builds candidate relaunch paths, and writes both a PowerShell launcher script and a small CMD wrapper. The PowerShell script waits for the current process, stops lingering builder processes, runs the installer, retries with elevation if necessary, waits for the installed executable, and relaunches it.

The backend launches the CMD wrapper detached, emits an update-started event, and exits the current process after a short delay. That sequence lets the app replace itself without asking the running executable to overwrite files that are still in use.

handleApi

handleApi is the local API router. It receives the parsed URL and checks specific method/path combinations in order. GET routes handle catalog reads, template reads, publish-target status, system checks, app updates, security reports, and compiler-tool status.

Full source for handleApi

```

async function handleApi(request, response, url) {
  if (request.method === "GET" && url.pathname === "/api/catalog") {
    try {
      const catalog = await readJsonFile(draftPath);
      sendJson(response, 200, { source: "draft", catalog });
    } catch {
      sendJson(response, 200, { source: "published", catalog: await readJsonFile(catalogPath) });
    }
    return true;
  }

  if (request.method === "GET" && url.pathname === "/api/templates") {
    sendJson(response, 200, await readJsonFile(path.join(root, "project-templates.json")));
    return true;
  }

  if (request.method === "GET" && url.pathname === "/api/publish-target") {
    if (!isLocalRequest(request)) {
      sendJson(response, 403, { error: "Publishing target details are only available from this computer." });
      return true;
    }
    sendJson(response, 200, { ok: true, target: await getPublishTargetInfo() });
    return true;
  }

  if (request.method === "GET" && url.pathname === "/api/system-check") {
    sendJson(response, 200, { ok: true, system: await getGitStatus(), target: await getPublishTargetInfo() });
    return true;
  }

  if (request.method === "GET" && url.pathname === "/api/app-update") {
    sendJson(response, 200, await getUpdateInfo());
    return true;
  }

  if (request.method === "GET" && url.pathname === "/api/security-report") {
    if (!isLocalRequest(request)) {
      sendJson(response, 403, { error: "Security reports are only available from this computer." });
      return true;
    }
    sendJson(response, 200, await getSecurityReport());
    return true;
  }
}

```

```

}

if (request.method === "GET" && url.pathname === "/api/code/tools") {
  if (!isLocalRequest(request)) {
    sendJson(response, 403, { error: "Compiler details are only available from this computer." });
    return true;
  }
  sendJson(response, 200, { ok: true, tools: await compileToolStatus() });
  return true;
}

if (request.method !== "POST") return false;

if (!isLocalRequest(request)) {
  sendJson(response, 403, { error: "Write actions are only allowed from this computer." });
  return true;
}

if (url.pathname === "/api/portfolio-ai") {
  await handlePortfolioAi(request, response);
  return true;
}

if (url.pathname === "/api/code/beautify") {
  try {
    const body = await readRequestJson(request);
    const language = normalizeCodeLanguage(body.language || detectCodeLanguageFromSource(body.code,
body.fileName));
    sendJson(response, 200, {
      ok: true,
      language,
      code: beautifyCode(body.code || "", language)
    });
  } catch (error) {
    sendJson(response, 400, { ok: false, error: error.message || "Code could not be beautified." });
  }
  return true;
}

if (url.pathname === "/api/code/save") {
  try {
    const body = await readRequestJson(request);
    const saved = await saveCompileSource(body);
    sendJson(response, 200, { ok: true, saved });
  } catch (error) {
    sendJson(response, 400, { ok: false, error: error.message || "Code could not be saved." });
  }
  return true;
}

if (url.pathname === "/api/code/compile") {
  try {
    const body = await readRequestJson(request);
    const result = await compileAndRunCode(body);
    sendJson(response, 200, { ok: result.ok, result });
  } catch (error) {
    sendJson(response, 400, { ok: false, error: error.message || "Code could not be compiled.", result: null });
  }
  return true;
}

if (url.pathname === "/api/code/install-tools") {
  try {
    const body = await readRequestJson(request);
    const result = await installCompilerTools(body.language || "");
    sendJson(response, 200, result);
  } catch (error) {
    sendJson(response, 400, { ok: false, error: error.message || "Compiler tools could not be installed." });
  }
  return true;
}

if (url.pathname === "/api/publish-target") {
  await readRequestJson(request).catch(() => ({}));
  sendJson(response, 400, {

```

```

    ok: false,
    error: "Publishing target setup now requires GitHub authentication.",
    details: "Use Authenticate target. The builder keeps the previous target until GitHub write access is
verified."
  });
  return true;
}

if (url.pathname === "/api/install-git") {
  try {
    const install = await installGitForWindows();
    sendJson(response, 200, { ok: true, install, system: await getGitStatus() });
  } catch (error) {
    sendJson(response, 400, {
      ok: false,
      error: error.message || "Git for Windows installer could not be started.",
      downloadUrl: "https://git-scm.com/download/win"
    });
  }
  return true;
}

if (url.pathname === "/api/app-update/install") {
  try {
    await readRequestJson(request).catch(() => ({}));
    const update = await downloadAndLaunchAppUpdate();
    sendJson(response, 200, { ok: true, update });
  } catch (error) {
    sendJson(response, 400, {
      ok: false,
      error: error.message || "The update could not be started."
    });
  }
  return true;
}

if (url.pathname === "/api/github-authenticate") {
  try {
    const body = await readRequestJson(request);
    const auth = await authenticateGitHubForTarget(body || {});
    sendJson(response, 200, { ok: true, auth, target: await getPublishTargetInfo() });
  } catch (error) {
    sendJson(response, 400, {
      ok: false,
      error: error.message || "GitHub authentication did not complete.",
      details: error.details || publishAuthorizationHelp,
      publishAccess: error.publishAccess || null,
      system: await getGitStatus()
    });
  }
  return true;
}

if (url.pathname === "/api/sync-from-publish-target") {
  try {
    const sync = await syncFromPublishTarget();
    sendJson(response, 200, { ok: true, sync, target: await getPublishTargetInfo() });
  } catch (error) {
    sendJson(response, 400, {
      ok: false,
      error: error.message || "Publishing target could not be imported.",
      details: error.details || publishAuthorizationHelp,
      publishAccess: error.publishAccess || null
    });
  }
  return true;
}

if (url.pathname === "/api/save-draft" || url.pathname === "/api/apply-catalog") {
  const body = await readRequestJson(request);
  const catalog = body.catalog;

  if (!catalog || !Array.isArray(catalog.categories) || !Array.isArray(catalog.projects) || !
catalog.categories.length) {
    sendJson(response, 400, { error: "Catalog must include categories and projects arrays." });
    return true;
  }

```

```

}

const applyingToSite = url.pathname === "/api/apply-catalog";
let publishAccess = null;
if (applyingToSite) {
  try {
    publishAccess = await assertPublishAccess();
  } catch (error) {
    sendJson(response, 200, {
      ok: false,
      file: path.relative(root, catalogPath),
      publish: {
        pushed: false,
        authorizationRequired: true,
        error: error.message || "Publishing authorization failed.",
        details: error.details || publishAuthorizationHelp,
        ...(error.publishAccess || {})
      }
    });
    return true;
  }
}

const target = applyingToSite ? catalogPath : draftPath;
await writeFile(target, `${JSON.stringify(catalog, null, 2)}\n`, "utf8");
if (applyingToSite) {
  try {
    const publish = await publishSiteChanges(publishAccess);
    sendJson(response, 200, { ok: true, file: path.relative(root, target), publish });
  } catch (error) {
    sendJson(response, 200, {
      ok: true,
      file: path.relative(root, target),
      publish: {
        pushed: false,
        error: error.stderr || error.stdout || error.message || "Git push failed."
      }
    });
  }
  return true;
}
sendJson(response, 200, { ok: true, file: path.relative(root, target) });
return true;
}

if (url.pathname === "/api/upload") {
  const body = await readRequestJson(request);
  const projectId = safeSegment(body.projectId, "project");
  const section = safeSegment(body.section, "documents");
  const fileName = safeFileName(body.fileName);
  const base64 = String(body.data || "").replace(/^data:[^;]+;base64/, "");

  if (!base64) {
    sendJson(response, 400, { error: "Upload data is missing." });
    return true;
  }

  const folder = resolveInsideRoot("docs", projectId, section);
  const filePath = resolveInsideRoot("docs", projectId, section, fileName);
  await mkdir(folder, { recursive: true });
  await writeFile(filePath, Buffer.from(base64, "base64"));

  sendJson(response, 200, {
    ok: true,
    url: path.relative(root, filePath).replaceAll(path.sep, "/")
  });
  return true;
}

return false;
}

createServer(async (request, response) => {
  try {
    const url = new URL(request.url || "/", `http://${request.headers.host}`);

```

```

if (url.pathname.startsWith("/api/") && await handleApi(request, response, url)) return;

const pathname = url.pathname === "/" ? "/index.html" : decodeURIComponent(url.pathname);
const filePath = path.normalize(path.join(root, pathname));

if (!filePath.startsWith(root)) {
  response.writeHead(403);
  response.end("Forbidden");
  return;
}

const body = await readFile(filePath);
response.writeHead(200, {
  ...securityHeaders(),
  "Content-Type": types[path.extname(filePath)] || "application/octet-stream",
  "Cache-Control": "no-store, no-cache, must-revalidate"
});
response.end(body);
} catch {
  response.writeHead(404);
  response.end("Not found");
}
}).listen(port, host, () => {
  console.log(`Portfolio preview running at http://localhost:${port}`);
  console.log("For phone access, use this computer's Wi-Fi/LAN IP address with the same port.");
});

```

After the read-only routes, the function rejects non-POST requests that were not handled. Then it applies the most important boundary rule in the API: POST write actions must come from the local machine. Only after that local check does it route AI requests, code beautify/save/compile/install actions, Git installation, app update installation, GitHub authentication, load from target, save draft, apply catalog, and uploads.

The function is large because it is replacing a framework router. The chronological ordering is the safety design: cheap read routes first, local-only write gate second, then each command branch wrapped in endpoint-specific try/catch logic so the frontend receives JSON rather than a crashed server.